LPI · LINUX ESSENTIALS · 010-160 · v1.6

جلد دوم

لینوکس
مقدماتی
LINUX ESSENTIALS



جلد دوم · از خط فرمان تا مدیریت سیستم

Pipe و Redirect، grep و Regex، آرشیو و فشرده‌سازی، پردازش متن با cut و sort و sed و awk، ویرایشگرهای nano و vim، اسکریپت‌نویسی Bash، سخت‌افزار و FHS، فرایند و سرویس و لاگ، شبکه و netplan، کاربر و گروه، sudo و مجوزها؛ همه با سوییچ‌های رایج، مثال واقعی و لابراتوار گام‌به‌گام.

```
student@ubuntu-lab: ~

student@ubuntu-lab:~$ grep -cE 'Failed|error' /var/log/auth.log
3
student@ubuntu-lab:~$ tar -czf backup-$(date +%F).tar.gz project
student@ubuntu-lab:~$ systemctl is-active ssh
active
student@ubuntu-lab:~$ ls -ld /srv/shared
drwxr-x--- 2 root devs 4.0K shared
student@ubuntu-lab:~$
```

TOPIC 3
قدرت خط فرمان (بخش دوم)

TOPIC 4
سیستم‌عامل و شبکه

TOPIC 5
امنیت و مجوزها

UBUNTU 24.04 LTS
VMware Workstation 17+
LPI Linux Essentials 1.6

مدرس و طراح لابراتوار

محمدرضا بیدختی

MOHAMMAD REZA BEIDOKHTI

EXAM 010-160 · TOPIC 2

2

پیدا کردن راه در سیستم لینوکس (ادامه)

FINDING YOUR WAY ON A LINUX SYSTEM · PART 2

در جلد اول یاد گرفتیم در درخت فایل‌ها راه برویم، فایل بسازیم و پیدا کنیم. حالا وقت آن است که خروجی فرمان‌ها را به هم وصل کنیم: داده را از یک فرمان بگیریم، به فرمان دیگر بدهیم، در فایل ذخیره کنیم و با grep و عبارات منظم فقط همان خطی را بیرون بکشیم که لازم داریم.

وزن در آزمون: ۶ سؤال از ۴۰

Pipe و Redirect: هدایت جریان داده	2.4
grep و عبارات منظم (Regex)	2.5
استخراج داده از لاگ‌های سیستم	LAB 10
جستجو در لاگ‌های SSH و سیستم	LAB 11

Objective 2.4

وزن در آزمون: ۳

Pipe و Redirect: هدایت جریان داده

Pipes and I/O Redirection

هر فرمانی که در ترمینال اجرا می‌کنید، مثل یک دستگاه کوچک در کارخانه است: از یک طرف مواد اولیه (ورودی) می‌گیرد و از طرف دیگر محصول (خروجی) تحویل می‌دهد. اگر مشکلی پیش بیاید، پیام خطا را از یک خروجی جداگانه بیرون می‌دهد. لینوکس به این سه مسیر **جریان‌های استاندارد** (Standard Streams) می‌گوید.

قدرت واقعی خط فرمان از اینجا می‌آید که این مسیرها را می‌توانید **جابه‌جا** کنید: خروجی را به‌جای صفحه در فایل بریزید، ورودی را به‌جای صفحه‌کلید از فایل بخوانید و از همه مهم‌تر، خروجی یک فرمان را مستقیم به ورودی فرمان بعدی بدهید. به این کار **Redirect** و **Pipe** می‌گوییم.

stdin (0)	ورودی استاندارد؛ به‌طور پیش‌فرض صفحه‌کلید. شماره‌ی توصیفگر (File Descriptor) آن `0` است.
stdout (1)	خروجی استاندارد؛ نتیجه‌ی عادی فرمان که به‌طور پیش‌فرض روی صفحه چاپ می‌شود. توصیفگر `1` .
stderr (2)	خروجی خطا؛ پیام‌های خطا و هشدار. آن هم روی صفحه می‌آید، اما مسیرش از stdout جداست. توصیفگر `2` .
Redirect	تغییر مقصد یا مبدأ یک جریان به فایل، با عملگرهایی مثل `>` و `<` .
Pipe	اتصال stdout یک فرمان به stdin فرمان بعدی با عملگر `|` .
dev/null/	«سیاه‌چاله»ی لینوکس؛ هر چه در آن بنویسید ناپدید می‌شود. برای دورریختن خروجی ناخواسته.

تشبیه

فرمان را یک شیر آب در نظر بگیرید که دو لوله‌ی خروجی دارد: لوله‌ی آب تمیز (stdout) و لوله‌ی فاضلاب (stderr). Redirect یعنی سر هر لوله را به یک سطل (فایل) ببرید. Pipe یعنی لوله‌ی آب تمیز را مستقیم به ورودی دستگاه تصفیه‌ی بعدی (فرمان بعدی) وصل کنید.

چرا stdout و stderr جدا هستند؟

فرض کنید فهرست همه‌ی فایل‌های `/etc` را در یک فایل ذخیره می‌کنید، اما به بعضی پوشه‌ها دسترسی ندارید. اگر خطاها و خروجی در یک مسیر بودند، فایل شما پر از پیام «Permission denied» می‌شد. چون مسیرها جدا هستند، می‌توانید خروجی تمیز را در فایل و خطاها را روی صفحه یا در فایل دیگری داشته باشید.

student@ubuntu-lab: ~

```
$ ls /etc/hostname /nothing
ls: cannot access '/nothing': No such file or directory
/etc/hostname
$ ls /etc/hostname /nothing > out.txt  ## only stdout goes to the file
ls: cannot access '/nothing': No such file or directory
$ cat out.txt
/etc/hostname
```

عملگرهای Redirect: بازنویسی و افزودن

Output Redirection Operators

> >>

SYNTAX · ساختار کلی

```
COMMAND >  FILE     # overwrite (stdout)
COMMAND >> FILE     # append (stdout)
COMMAND 2> FILE     # stderr to file
COMMAND <  FILE     # read stdin from file
```

عملگر `>` فایل مقصد را **از اول می‌سازد**؛ اگر فایل وجود داشته باشد، محتوای قبلی‌اش بدون هیچ پرسشی پاک می‌شود. عملگر `>>` خروجی را **به انتهای فایل اضافه می‌کند** و محتوای قبلی دست‌نخورده می‌ماند. عدد قبل از عملگر مشخص می‌کند کدام جریان هدایت شود؛ اگر عددی ننویسید، منظور `1` یعنی stdout است.

عملگر	معنی	مثال
`>`	stdout را در فایل بنویس (بازنویسی)	`date > now.txt`
`>>`	stdout را به انتهای فایل اضافه کن	`date >> log.txt`
`2>`	stderr را در فایل بنویس	`find / -name x 2> err.txt`
`2>>`	stderr را به انتهای فایل اضافه کن	`cmd 2>> err.log`
`&>`	هم stdout و هم stderr را در یک فایل بنویس	`cmd &> all.log`
`&>>`	هر دو را به انتهای فایل اضافه کن	`cmd &>> all.log`
`2>&1`	stderr را به همان جایی بفرست که stdout می‌رود	`cmd > all.log 2>&1`
`<`	stdin را از فایل بخوان	`wc -l < /etc/passwd`
`<<`	Here Document: چند خط ورودی تا رسیدن به نشانه‌ی پایان	`cat << EOF`
`<<<`	Here String: یک رشته به‌عنوان ورودی	`wc -w <<< "a b c"`

student@ubuntu-lab: ~

```
$ echo "first line" > notes.txt
$ echo "second line" >> notes.txt
$ cat notes.txt
first line
second line
$ echo "oops" > notes.txt  ## overwrite!
$ cat notes.txt
oops
```

هشدار

عملگر `>` بی‌صدا فایل را خالی می‌کند. اشتباه معروف: `sort file.txt > file.txt` . شِل **قبل از اجرای** sort فایل را خالی می‌کند و sort چیزی برای خواندن ندارد؛ نتیجه یک فایل خالی است. همیشه در فایل جدید بنویسید و بعد آن را جابه‌جا کنید. با فرمان `set -o noclobber` می‌توانید شِل را مجبور کنید روی فایل موجود ننویسد.

هدایت خطاها و دورریختن خروجی

stderr, 2>&1 and /dev/null

&>

وقتی فرمانی مثل `find` در کل دیسک جستجو می‌کند، صدها پیام «Permission denied» می‌دهد که نتیجه‌ی اصلی را گم می‌کند. سه راه رایج برای کنترل این پیام‌ها داریم:

student@ubuntu-lab: ~

```
$ find / -name "*.conf" 2> /dev/null  ## throw errors away
$ find / -name "*.conf" > found.txt 2> errors.txt  ## split into two files
$ find / -name "*.conf" &> all.txt  ## both streams in one file (bash)
$ find / -name "*.conf" > all.txt 2>&1  ## same thing, POSIX style
$ wc -l found.txt errors.txt
  137 found.txt
   52 errors.txt
  189 total
```

در عبارت `> all.txt 2>&1` ترتیب مهم است. شِل از چپ به راست می‌خواند: اول stdout را به فایل می‌برد، بعد می‌گوید «stderr را به همان جایی ببر که الان stdout می‌رود». اگر ترتیب را برعکس بنویسید (`2>&1 > all.txt`)، stderr به صفحه‌ی قبلی وصل می‌ماند و فقط stdout در فایل می‌رود.

؟ نکته‌ی آزمون

در آزمون زیاد می‌پرسند کدام عبارت **هم خروجی و هم خطا** را در فایل می‌ریزد. پاسخ‌های درست: `&> file` و `> file 2>&1` . عبارت `2> file` فقط خطاها را ذخیره می‌کند. همچنین بدانید `/dev/null` فایل ویژه‌ای است که هر چه در آن بنویسید دور ریخته می‌شود.

Here Document: چند خط ورودی در یک فرمان

گاهی می‌خواهید چند خط متن را مستقیم به یک فرمان بدهید یا فایلی کوچک بسازید، بدون اینکه ویرایشگر باز کنید. عملگر `<<` تا رسیدن به کلمه‌ی پایان (معمولاً `EOF`) همه‌ی خطوط را به‌عنوان stdin می‌فرستد. این روش در اسکریپت‌ها بسیار رایج است.

student@ubuntu-lab: ~

```
$ cat << EOF > servers.txt
> web01 192.168.56.11
> db01  192.168.56.12
> EOF
$ cat servers.txt
web01 192.168.56.11
db01  192.168.56.12
$ wc -w <<< "one two three"  ## here string
3
```

Pipe: زنجیره‌ی فرمان‌ها

Building Command Pipelines

SYNTAX · ساختار کلی

```
COMMAND1 | COMMAND2 | COMMAND3 ...
```

فلسفه‌ی Unix می‌گوید: «هر برنامه یک کار را خوب انجام دهد و برنامه‌ها با هم کار کنند». Pipe همین «با هم کار کردن» است. خروجی استاندارد فرمان سمت چپ، بدون ساختن فایل موقت، مستقیم وارد ورودی استاندارد فرمان سمت راست می‌شود. هر تعداد فرمان را می‌توانید پشت هم وصل کنید.

student@ubuntu-lab: ~

```
$ ls /etc | wc -l  ## how many entries in /etc?
215
$ cat /etc/passwd | head -3
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
$ ps aux | sort -k4 -nr | head -5  ## top 5 memory users
$ history | grep apt | tail -3  ## last 3 apt commands
```

نکته

Pipe فقط stdout را منتقل می‌کند، نه stderr را. اگر می‌خواهید خطاها هم وارد Pipe شوند، از `2>&1 |` یا شکل کوتاه آن در Bash یعنی `|&` استفاده کنید: `make |& less` .

ترکیب Pipe با Redirect

می‌توانید در انتهای یک زنجیره، نتیجه را در فایل ذخیره کنید. در مثال زیر کاربرانی را که شِل Bash دارند پیدا می‌کنیم، مرتب می‌کنیم و در فایل می‌نویسیم:

student@ubuntu-lab: ~

```
$ grep "/bin/bash$" /etc/passwd | cut -d: -f1 | sort > bash-users.txt
$ cat bash-users.txt
root
student
```

تشبیه

Pipe مثل خط تولید است: اولی آرد را الک می‌کند، دومی خمیر می‌زند، سومی می‌پزد. هیچ‌کدام از کارگرها لازم نیست بدانند بقیه چه می‌کنند؛ فقط از سینی قبلی برمی‌دارند و در سینی بعدی می‌گذارند.

نکته‌ی آزمون

فرق `|` و `<` را با دقت بدانید: `|` خروجی را به **یک فرمان** می‌دهد و `<` خروجی را در **یک فایل** می‌نویسد. عبارت `ls > wc` فایلی به نام wc می‌سازد و هیچ شمارشی انجام نمی‌دهد!

تقسیم خروجی با tee

Splitting Output with tee

tee

SYNTAX · ساختار کلی

```
COMMAND | tee [-a] FILE... | NEXT_COMMAND
```

گاهی می‌خواهید خروجی یک فرمان را **هم روی صفحه ببینید و هم در فایل ذخیره کنید**. فرمان `tee` مثل یک سه‌راهی لوله‌کشی عمل می‌کند: stdin را می‌گیرد، یک نسخه را در فایل (یا فایل‌ها) می‌نویسد و همان را به stdout می‌فرستد تا زنجیره ادامه پیدا کند.

سوییچ‌های رایج `tee`

سوییچ	شکل بلند	کاربرد	مثال
`-a`	`--append`	به انتهای فایل اضافه کن، بازنویسی نکن	`ls | tee -a list.txt`
`-i`	`--ignore-interrupts`	با Ctrl+C متوقف نشو	`cmd | tee -i out.log`
`-p`	`--output-error`	در صورت خطای نوشتن، گزارش بده	`cmd | tee -p out.log`

student@ubuntu-lab: ~

```
$ df -h | tee disk-report.txt  ## see it AND save it
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        30G  9.1G   19G  33% /
$ uname -r | tee -a disk-report.txt
6.8.0-45-generic
$ echo "nameserver 1.1.1.1" | sudo tee -a /etc/resolv-test.conf
nameserver 1.1.1.1
```

نکته

ترفند حرفه‌ای: عبارت `sudo echo x > /etc/file` کار نمی‌کند، چون Redirect را شِل خودتان (بدون دسترسی root) انجام می‌دهد، نه sudo. راه درست: `echo x | sudo tee /etc/file` . در این حالت `tee` با دسترسی root فایل را می‌نویسد. اگر خروجی را روی صفحه نمی‌خواهید، `> /dev/null` به انتها اضافه کنید.

نکته‌ی آزمون

سؤال رایج: «کدام فرمان خروجی را هم روی صفحه نشان می‌دهد و هم در فایل ذخیره می‌کند؟» پاسخ: `tee` . سوییچ `-a` آن معادل `>>` است.

LAB 10

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

استخراج داده از لاگ‌های سیستم

با Redirect و Pipe از فایل‌های لاگ واقعی Ubuntu گزارش‌های کوچک بسازید، خطاها را جدا کنید و نتیجه را هم ببینید و هم ذخیره کنید.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. پوشه‌ی `~/lab10` را بسازید و وارد آن شوید.

۲. تعداد خطوط فایل `/var/log/syslog` را بشمارید، بدون اینکه نام فایل در خروجی بیاید.

۳. ده خط آخر syslog را در فایل `last10.txt` ذخیره کنید و سپس تاریخ امروز را به انتهای همان فایل اضافه کنید.

۴. در کل `/etc` دنبال فایل‌های `*.conf` بگردید؛ نتیجه را در `conf.txt` و خطاها را در `err.txt` بریزید.

۵. همین جستجو را تکرار کنید و هر دو جریان را در `all.txt` ذخیره کنید.

۶. فهرست ۵ فایل بزرگ `/var/log` را با Pipe بسازید و با `tee` در `big-logs.txt` هم ذخیره کنید.

۷. با Here Document یک فایل گزارش سه‌خطی بسازید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ mkdir -p ~/lab10 && cd ~/lab10
$ wc -l < /var/log/syslog  ## input redirect: no filename in output
4821
$ tail -n 10 /var/log/syslog > last10.txt
$ date >> last10.txt
$ tail -n 2 last10.txt
Oct  4 12:01:07 ubuntu-lab systemd[1]: Started session-3.scope
Sun Oct  4 12:02:15 PM +0330 2026
```

student@ubuntu-lab: ~

```
$ find /etc -name "*.conf" > conf.txt 2> err.txt
$ wc -l conf.txt err.txt
  137 conf.txt
    3 err.txt
  140 total
$ find /etc -name "*.conf" &> all.txt
$ wc -l all.txt
140 all.txt
```

student@ubuntu-lab: ~

```
$ sudo du -ah /var/log 2>/dev/null | sort -rh | head -5 | tee big-logs.txt
61M  /var/log
38M  /var/log/journal
2.1M /var/log/syslog
1.2M /var/log/kern.log
640K /var/log/dpkg.log
$ cat << EOF > report.txt
> Host: $(hostname)
> User: $(whoami)
> Date: $(date +%F)
> EOF
$ cat report.txt
Host: ubuntu-lab
User: student
Date: 2026-10-04
```

✓ بررسی نتیجه (Verify)

فایل `conf.txt` باید فقط مسیرهای فایل داشته باشد و `err.txt` فقط پیام‌های «Permission denied». جمع خطوط این دو برابر با تعداد خطوط `all.txt` است. اگر syslog وجود نداشت، دستور `sudo apt install rsyslog` را بزنید یا از `journalctl` استفاده کنید (درس ۴/۴).

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. فرق `>` و `>>` چیست و کدام‌یک ممکن است داده را از بین ببرد؟
2. چرا `wc -l < file` نام فایل را چاپ نمی‌کند ولی `wc -l file` چاپ می‌کند؟
3. دو روش برای ذخیره‌ی هم‌زمان stdout و stderr در یک فایل بنویسید.
4. چرا `sudo echo test > /etc/x` با خطای دسترسی روبه‌رو می‌شود و راه درست چیست؟
5. خروجی فرمان `ls /etc | wc -l` چه چیزی را نشان می‌دهد؟

وزن در آزمون: ۳

Objective 2.5

grep و عبارات منظم (Regex)

Searching Text with grep and Regular Expressions

روی هر سرور واقعی، هزاران خط لاگ و صدها فایل تنظیمات وجود دارد. مدیر سیستم هرگز این‌ها را از اول تا آخر نمی‌خواند؛ با `grep` فقط خطوطی را بیرون می‌کشد که به کارش می‌آید. نام grep از فرمان قدیمی ویرایشگر ed آمده: `g/re/p` یعنی «در همه‌جا (global) دنبال عبارت منظم (regular expression) بگرد و چاپ (print) کن».

عبارت منظم (Regular Expression یا Regex) زبان کوچکی برای توصیف الگوی متن است. به‌جای جستجوی یک کلمه‌ی ثابت، می‌توانید بگویید «خطی که با عدد شروع شود»، «هر آدرس IP» یا «کلمه‌ای که با error یا fail تمام شود».

الگو (Pattern)	متنی که دنبالش می‌گردیم؛ می‌تواند ساده (`ssh`) یا یک Regex (`^ssh.*d$`) باشد.
Literal	نویسه‌ای که معنای خودش را دارد؛ مثل حروف و اعداد.
Metacharacter	نویسه‌ی ویژه با معنای خاص؛ مثل `^` و `$` و `.` و `*`.
BRE	Basic Regex؛ حالت پیش‌فرض grep. در آن `+` و `?` و `|` باید با `\` نوشته شوند.
ERE	Extended Regex؛ با `grep -E` (یا `egrep`). نویسه‌های ویژه بدون بک‌اسلش کار می‌کنند.
Anchor	لنگر؛ مکان را مشخص می‌کند نه نویسه را: `^` آغاز خط و `$` پایان خط.

تشبیه

grep مثل یک آهن‌ربا است که از روی یک کیسه پر از خط‌های متن رد می‌شود و فقط خط‌هایی را که «فلز» دارند (با الگو جور هستند) بیرون می‌کشد. Regex نوع آهن‌ربا را مشخص می‌کند: فقط آهن، یا هر فلزی که با «م» شروع شود.

هشدار

همیشه الگوی Regex را بین **نقل‌قول تکی** بگذارید: `grep '^#' file` . بدون نقل‌قول، Bash نویسه‌هایی مثل `*` و `$` و `?` را قبل از grep تفسیر می‌کند (Globbing و متغیرها) و نتیجه کاملاً غیرمنتظره می‌شود.

فرمان grep و سوییچ‌های اصلی
The grep Command

grep

SYNTAX · ساختار کلی
```
grep [OPTIONS] 'PATTERN' [FILE...]
COMMAND | grep [OPTIONS] 'PATTERN'
```

سوییچ‌های رایج grep

سوییچ	شکل بلند	کاربرد	مثال
`-i`	`--ignore-case`	بی‌توجه به حروف کوچک و بزرگ	`grep -i error syslog`
`-v`	`--invert-match`	خطوطی که الگو را **ندارند**	`grep -v '^#' file`
`-n`	`--line-number`	شماره‌ی خط را هم نشان بده	`grep -n root /etc/passwd`
`-c`	`--count`	فقط تعداد خطوط منطبق	`grep -c bash /etc/passwd`
`-r`	`--recursive`	جستجوی بازگشتی در پوشه	`grep -r Port /etc/ssh`
`-l`	`--files-with-matches`	فقط نام فایل‌های منطبق	`grep -rl ubuntu /etc`
`-w`	`--word-regexp`	فقط کلمه‌ی کامل	`grep -w ssh services`
`-o`	`--only-matching`	فقط بخش منطبق، نه کل خط	`grep -o '[0-9]*' f`
`-E`	`--extended-regexp`	Regex گسترده (ERE)	`grep -E 'a|b' f`
`-F`	`--fixed-strings`	الگو را متن ثابت بدان	`grep -F 'a.b*' f`
`-A n`	`--after-context`	n خط بعد از هر تطبیق	`grep -A2 Failed auth.log`
`-B n`	`--before-context`	n خط قبل از هر تطبیق	`grep -B1 error log`
`--color`	`--color=auto`	رنگی‌کردن بخش منطبق	`grep --color ssh f`

student@ubuntu-lab: ~

```
$ grep -n student /etc/passwd
33:student:x:1000:1000:student,,,:/home/student:/bin/bash
$ grep -c nologin /etc/passwd
27
$ grep -v '^#' /etc/ssh/sshd_config | grep -v '^$'  ## config without comments & blanks
Include /etc/ssh/sshd_config.d/*.conf
KbdInteractiveAuthentication no
UsePAM yes
```

نکته

ترکیب طلایی برای خواندن فایل تنظیمات: `grep -Ev '^\s*(#|$)' file` . این عبارت همه‌ی خطوط توضیح و خطوط خالی را حذف می‌کند و فقط تنظیمات فعال می‌ماند.

عبارات منظم: لنگرها و نویسه‌ی هر چیز

Anchors and the Dot

^ $.

سه نویسه‌ی ویژه‌ی اول، پایه‌ی هر Regex هستند. `^` یعنی «آغاز خط»، `$` یعنی «پایان خط» و `.` یعنی «دقیقاً یک نویسه، هر چه باشد». اگر بخواهید خود نقطه را جستجو کنید، باید آن را با بک‌اسلش «فرار» دهید: `\.`

الگو	معنی	منطبق می‌شود با	منطبق نمی‌شود با
`^root`	خطی که با root شروع شود	`root:x:0:0...`	`chroot` (وسط خط)
`bash$`	خطی که با bash تمام شود	`.../bin/bash`	`bash-completion`
`^$`	خط خالی	(خط بدون هیچ نویسه)	(فاصله دارد)
`^#`	خط توضیح	`#Port 22`	`Port 22 #x`
`h.t`	h، یک نویسه‌ی دلخواه، t	`hat` ، `hot` ، `h-t`	`ht` ، `heat`
`\.conf$`	پایان با .conf واقعی	`ssh.conf`	`sshconf`
`^...$`	خطی با دقیقاً ۳ نویسه	`abc` ، `1 2`	`ab` ، `abcd`

student@ubuntu-lab: ~

```
$ grep '^root' /etc/passwd
root:x:0:0:root:/root:/bin/bash
$ grep 'bash$' /etc/passwd
root:x:0:0:root:/root:/bin/bash
student:x:1000:1000:student,,,:/home/student:/bin/bash
$ grep -c '^$' /etc/ssh/sshd_config  ## count empty lines
19
$ ls /etc | grep '\.conf$' | head -3
adduser.conf
ca-certificates.conf
debconf.conf
```

نکته‌ی آزمون

فرق Globbing و Regex را قاطی نکنید: در Globbing (نام فایل در شِل) `*` یعنی «هر چیزی» و `?` یعنی «یک نویسه». در Regex، `.` یعنی «یک نویسه» و `*` یعنی «تکرار نویسه‌ی **قبلی** صفر بار یا بیشتر». پس معادل Regex برای glob `*.txt` عبارت `.*\.txt$` است.

کلاس نویسه‌ها، بازه‌ها و نفی

Character Classes and Ranges

[ ]

براکت [ ] یعنی «**یکی** از نویسه‌های داخل این مجموعه». داخل براکت می‌توانید بازه بنویسید (a-z) یا با ^ در **ابتدای** براکت مجموعه را نفی کنید. توجه کنید ^ بیرون از براکت یعنی آغاز خط و داخل براکت یعنی «به‌جز».

الگو	معنی	مثال تطبیق
[abc]	یکی از a یا b یا c	gray، grey → gr[ae]y
[a-z]	یک حرف کوچک انگلیسی	[a-z] در x
[A-Z]	یک حرف بزرگ	^[A-Z] خطوطی که با حرف بزرگ شروع شوند
[0-9]	یک رقم	[0-9][0-9] دو رقم پشت هم
[a-zA-Z0-9_]	حرف، رقم یا زیرخط	نام متغیرها
[^0-9]	هر نویسه‌ای **به‌جز** رقم	a ، - ، فاصله
[^#]	هر نویسه‌ای به‌جز #	^[^#] خطوطی که توضیح نیستند
[[:digit:]]	کلاس POSIX رقم (مثل [0-9])	[[:digit:]]+
[[:space:]]	فاصله، Tab و…	^[[:space:]]*$ خط سفید
[[:alpha:]]	حرف	[[:upper:]] حرف بزرگ

student@ubuntu-lab: ~

```
$ grep '^[A-Z]' /etc/ssh/sshd_config | head -3  ## active settings start with capital
Include /etc/ssh/sshd_config.d/*.conf
KbdInteractiveAuthentication no
UsePAM yes
$ grep -c '^[^#]' /etc/services
343
$ echo "gray grey graay" | grep -o 'gr[ae]y'
gray
grey
```

نکته

کلاس‌های POSIX مثل [[:digit:]] و [[:space:]] دو براکت دارند: براکت بیرونی یعنی «مجموعه» و [:digit:] نام کلاس است. این کلاس‌ها در زبان‌های غیرانگلیسی (Locale) هم درست کار می‌کنند و خواناتر از بازه‌ها هستند.

تکرار، انتخاب و تعداد دقیق

Quantifiers, Alternation and Intervals

* + ?

کمیت‌نماها (Quantifiers) می‌گویند نویسه یا گروهِ **قبل از خودشان** چند بار تکرار شود. در grep معمولی (BRE) فقط `*` بدون بک‌اسلش کار می‌کند؛ برای `+` و `?` و `|` و `{ }` بدون بک‌اسلش از `grep -E` استفاده کنید. پرانتز `( )` چند نویسه را یک گروه می‌کند.

الگو (ERE)	معنی	مثال	تطبیق
`*`	صفر بار یا بیشتر	`ab*c`	ac، abc، abbbc
`+`	یک بار یا بیشتر	`ab+c`	abc، abbc (نه ac)
`?`	صفر یا یک بار (اختیاری)	`colou?r`	color، colour
`|`	یا (انتخاب)	`error|fail`	هر کدام از دو کلمه
`{n}`	دقیقاً n بار	`[0-9]{3}`	۳ رقم پشت هم
`{n,m}`	بین n تا m بار	`[0-9]{1,3}`	۱ تا ۳ رقم
`{n,}`	حداقل n بار	`a{2,}`	aa، aaa، ...
`( )`	گروه‌بندی	`(ab)+`	ab، abab
`.*`	هر چیزی با هر طولی	`^Port.*`	کل خط Port

student@ubuntu-lab: ~

```
$ grep -E 'error|fail|denied' /var/log/syslog | wc -l
42
$ grep -E '^(root|student):' /etc/passwd
root:x:0:0:root:/root:/bin/bash
student:x:1000:1000:student,,,:/home/student:/bin/bash
$ ip addr | grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}'  ## IPv4 addresses
127.0.0.1
192.168.17.128
192.168.56.128
$ grep 'colou\?r' text.txt  ## BRE needs backslash
$ grep -E 'colou?r' text.txt  ## ERE: same result
```

؟ نکته‌ی آزمون

نکات پرتکرار: `grep -E` همان `egrep` قدیمی و `grep -F` همان `fgrep` است. `.*` یعنی «هر چیزی». الگوی `^$` خطوط خالی است و `grep -v '^$'` آنها را حذف می‌کند. `-c` تعداد **خطوط** را می‌شمارد، نه تعداد تکرار کلمه.

≈ تشبیه

کمیت‌نماها مثل دستور پخت هستند: «شکر؟» یعنی اگر خواستی کمی بریز، «نمک+» یعنی حداقل یک قاشق، «فلفل*» یعنی هر قدر خواستی، حتی هیچ، و «تخم‌مرغ{2}» یعنی دقیقاً دو تا.

LAB 11

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

جستجو و فیلتر کردن لاگ‌های SSH و سیستم

با grep و Regex تلاش‌های ورود ناموفق SSH را پیدا کنید، آدرس‌های IP را بیرون بکشید و فایل تنظیمات را بدون توضیحات بخوانید.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. مطمئن شوید سرویس SSH فعال است و با رمز اشتباه دو بار به `localhost` وصل شوید تا لاگ خطا تولید شود.

۲. تعداد خطوط حاوی `Failed password` را در `/var/log/auth.log` بشمارید.

۳. همان خطوط را با شماره‌ی خط و بدون حساسیت به حروف کوچک و بزرگ نمایش دهید.

۴. فقط آدرس‌های IP را از خطوط ناموفق بیرون بکشید.

۵. فایل `sshd_config` را بدون خطوط توضیح و خطوط خالی نمایش دهید.

۶. در کل `/etc` نام فایل‌هایی را که کلمه‌ی کامل `Port` دارند پیدا کنید.

۷. در syslog خطوطی را پیدا کنید که یکی از کلمات error یا warning یا fail را دارند.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ systemctl is-active ssh
active
$ ssh wronguser@localhost  ## type a wrong password twice, then Ctrl+C
$ sudo grep -c 'Failed password' /var/log/auth.log
2
$ sudo grep -in 'failed password' /var/log/auth.log
118:2026-10-04T12:10:31 ubuntu-lab sshd[2241]: Failed password for invalid user wronguser from
127.0.0.1 port 51522 ssh2
```

student@ubuntu-lab: ~

```
$
sudo grep 'Failed password' /var/log/auth.log | grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' | sort |
uniq -c
      2 127.0.0.1
$ grep -Ev '^\s*(#|$)' /etc/ssh/sshd_config
Include /etc/ssh/sshd_config.d/*.conf
KbdInteractiveAuthentication no
UsePAM yes
X11Forwarding yes
PrintMotd no
AcceptEnv LANG LC_*
Subsystem sftp /usr/lib/openssh/sftp-server
```

```
student@ubuntu-lab: ~
$ sudo grep -rlw 'Port' /etc 2>/dev/null
/etc/ssh/sshd_config
/etc/ssh/ssh_config
$ sudo grep -iE 'error|warning|fail' /var/log/syslog | tail -3
```

✓ بررسی نتیجه (Verify)

باید حداقل دو خط `Failed password` با آدرس `127.0.0.1` ببینید. خروجی فرمان sshd_config باید فقط چند خط تنظیم فعال باشد و هیچ خطی با `#` شروع نشود. اگر فایل auth.log خالی بود، از `sudo journalctl -u ssh | grep Failed` استفاده کنید.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. فرق `grep -v` و `grep -c` چیست؟
2. الگوی `^$` و الگوی `^#` به ترتیب چه خطوطی را پیدا می‌کنند؟
3. چرا `grep 'a+'` در حالت عادی با `grep -E 'a+'` نتیجه‌ی متفاوتی می‌دهد؟
4. یک Regex بنویسید که خطوطی را پیدا کند که با یک رقم شروع می‌شوند.
5. سوییچ‌های `-r` و `-l` و `-w` با هم چه می‌کنند؟

EXAM 010-160 · TOPIC 3

3

قدرت خط فرمان (بخش دوم)

THE POWER OF THE COMMAND LINE · PART 2

در این فصل از «کاربر ترمینال» به «سازنده‌ی ابزار» تبدیل می‌شوید: فایل‌ها را آرشیو و فشرده می‌کنید، متن را برش می‌دهید و مرتب و جایگزین می‌کنید، با nano و vim فایل تنظیمات ویرایش می‌کنید و در پایان اولین اسکریپت‌های Bash خودتان را می‌نویسید.

وزن در آزمون: ۹ سؤال از ۴۰

3.1	آرشیو و فشرده‌سازی
3.2	پردازش متن: cut، sort، uniq، wc، tr، sed، awk
3.3	ویرایشگرهای nano و vim
3.4	اسکریپت‌نویسی Bash
LAB 12-16	پنج لابراتوار عملی

وزن در آزمون: ۲

Objective 3.1

آرشیو و فشرده‌سازی فایل‌ها

Archiving Files on the Command Line

دو مفهوم را از هم جدا کنید: **آرشیو** یعنی چند فایل و پوشه را در یک فایل جمع کنیم (بدون کم‌کردن حجم)، و **فشرده‌سازی** یعنی حجم یک فایل را با الگوریتم کم کنیم. در لینوکس معمولاً این دو کار را جدا انجام می‌دهند: `tar` آرشیو می‌سازد و `gzip` یا `bzip2` یا `xz` آن را فشرده می‌کند. فایل `backup.tar.gz` یعنی «آرشیو tar که با gzip فشرده شده».

tar	Tape ARchiver؛ ابزار آرشیو. نام آن از نوارهای پشتیبان قدیمی آمده.
tarball	فایل آرشیو tar؛ معمولاً با پسوند `.tar` یا فشرده‌ی آن.
gzip (.gz)	سریع‌ترین و رایج‌ترین فشرده‌ساز؛ نسبت فشرده‌سازی متوسط.
bzip2 (.bz2)	کندتر از gzip اما فشرده‌تر.
xz (.xz)	کندترین در ساخت ولی بهترین نسبت فشرده‌سازی؛ بسته‌های کرنل با آن منتشر می‌شوند.
zip (.zip)	آرشیو و فشرده‌سازی در یک قدم؛ سازگار با Windows.

≈ تشبیه

آرشیو مثل جمع‌کردن وسایل اسباب‌کشی در یک کارتن است؛ تعداد وسایل کم نمی‌شود ولی حمل یک کارتن راحت‌تر است. فشرده‌سازی مثل کیسه‌ی وکیوم است که هوای لباس‌ها را می‌کشد تا جای کمتری بگیرند.

ابزار	فشرده‌سازی	بازکردن	پسوند	سوییچ tar
gzip	`gzip file`	`gunzip file.gz`	`.gz`	`-z`
bzip2	`bzip2 file`	`bunzip2 file.bz2`	`.bz2`	`-j`
xz	`xz file`	`unxz file.xz`	`.xz`	`-J`
zip	`zip -r a.zip dir`	`unzip a.zip`	`.zip`	–

student@ubuntu-lab: ~

```
$ cp /var/log/syslog ~/s.log; ls -lh ~/s.log
-rw-r----- 1 student student 2.1M Oct  4 12:20 s.log
$ gzip -k ~/s.log  ## -k keeps the original
$ ls -lh ~/s.log*
-rw-r----- 1 student student 2.1M Oct  4 12:20 /home/student/s.log
-rw-r----- 1 student student 214K Oct  4 12:20 /home/student/s.log.gz
$ zcat ~/s.log.gz | head -1  ## read without extracting
```

نکته

gzip و bzip2 و xz به‌طور پیش‌فرض فایل اصلی را **حذف** و نسخه‌ی فشرده را جایگزین می‌کنند. برای نگه‌داشتن اصل فایل از `-k` استفاده کنید. ابزارهای `zcat` ، `bzcat` و `xzcat` محتوای فایل فشرده را بدون بازکردن نشان می‌دهند و `zgrep` در آن جستجو می‌کند.

فرمان tar: ساخت، استخراج و فهرست

The tar Command

tar

SYNTAX · ساختار کلی

```
tar -c[z|j|J]vf ARCHIVE FILES...   # create
tar -x[z|j|J]vf ARCHIVE [-C DIR]     # extract
tar -t[z|j|J]vf ARCHIVE              # list
```

`tar` یک «فعل» اصلی می‌خواهد (`c` ساختن، `x` استخراج یا `t` فهرست) و تقریباً همیشه `f` که نام فایل آرشیو را بلافاصله بعد از خودش می‌گیرد. سوییچ فشرده‌سازی (`z`، `j` یا `J`) را هم به آن اضافه می‌کنید. ترتیب حروف مهم نیست، فقط `f` باید آخر باشد تا نام فایل بعد از آن بیاید.

سوییچ‌های رایج `tar`

سوییچ	شکل بلند	کاربرد	مثال
`-c`	`--create`	ساخت آرشیو جدید	`tar -cf a.tar dir/`
`-x`	`--extract`	استخراج آرشیو	`tar -xf a.tar`
`-t`	`--list`	فهرست محتوای آرشیو	`tar -tf a.tar`
`-f`	`--file`	نام فایل آرشیو (همیشه لازم)	`tar -cf NAME ...`
`-v`	`--verbose`	نمایش نام فایل‌ها حین کار	`tar -cvf a.tar d/`
`-z`	`--gzip`	فشرده‌سازی با gzip	`tar -czf a.tar.gz d/`
`-j`	`--bzip2`	فشرده‌سازی با bzip2	`tar -cjf a.tar.bz2 d/`
`-J`	`--xz`	فشرده‌سازی با xz	`tar -cJf a.tar.xz d/`
`-C`	`--directory`	استخراج در پوشه‌ی دیگر	`tar -xf a.tar -C /tmp`
`-p`	`--preserve-permissions`	حفظ مجوزها هنگام استخراج	`sudo tar -xpf a.tar`
	`--exclude`	نادیده گرفتن الگو	`--exclude='*.log'`
`-r`	`--append`	افزودن فایل به آرشیو ناتمام	`tar -rf a.tar new`

student@ubuntu-lab: ~

```
$ tar -czvf etc-ssh.tar.gz /etc/ssh 2>/dev/null
/etc/ssh/
/etc/ssh/ssh_config
/etc/ssh/sshd_config
...
$ tar -tzf etc-ssh.tar.gz | head -3  ## note: leading / was removed
etc/ssh/
etc/ssh/ssh_config
etc/ssh/sshd_config
$ mkdir restore && tar -xzf etc-ssh.tar.gz -C restore
$ ls restore/etc/ssh | head -2
moduli
ssh_config
```

هشدار

`tar` نوشتن روی فایل‌های موجود را بدون پرسش انجام می‌دهد. هرگز یک آرشیو ناشناس را در `/` یا پوشه‌ی خانه‌تان استخراج نکنید؛ اول با `tar -tf` ببینید داخلش چیست و بعد با `-C` در یک پوشه‌ی خالی بازش کنید.

نکته‌ی آزمون

پرسش‌های رایج آزمون: «کدام فرمان آرشیو `a.tar.bz2` را باز می‌کند؟» (`tar -xjf a.tar.bz2`)؛ «کدام سوییچ فقط محتوا را نشان می‌دهد؟» (`-t`)؛ «tar به‌تنهایی فشرده می‌کند یا نه؟» (نه، فقط با `z/j/J`). نسخه‌های جدید tar نوع فشرده‌سازی را هنگام استخراج خودشان تشخیص می‌دهند، پس `tar -xf` هم کار می‌کند.

فرمان zip و unzip برای تبادل با Windows

zip and unzip

zip

وقتی فایل را برای کاربر Windows می‌فرستید، `zip` انتخاب امن‌تری است، چون Windows بدون نرم‌افزار اضافه آن را باز می‌کند. برخلاف tar، zip آرشیو و فشرده‌سازی را در یک قدم انجام می‌دهد. اگر نصب نیست: `sudo apt install zip unzip` .

سوییچ‌های رایج `zip` و `unzip`

سوییچ	شکل بلند	کاربرد	مثال
`-r`	`--recurse-paths`	zip: افزودن پوشه با تمام زیرپوشه‌ها	`zip -r docs.zip docs/`
`-e`	`--encrypt`	zip: رمزگذاری با رمز عبور	`zip -er s.zip secret/`
`-9`		zip: بیشترین فشرده‌سازی	`zip -9r a.zip d/`
`-l`		unzip: فهرست محتوا	`unzip -l docs.zip`
`-d`		unzip: استخراج در پوشه	`unzip a.zip -d out/`
`-o`		unzip: بازنویسی بدون پرسش	`unzip -o a.zip`

student@ubuntu-lab: ~

```
$ zip -r docs.zip ~/Documents
  adding: home/student/Documents/ (stored 0%)
  adding: home/student/Documents/notes.txt (deflated 41%)
$ unzip -l docs.zip
Archive:  docs.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
        0  2026-10-04 12:30   home/student/Documents/
      812  2026-10-04 12:29   home/student/Documents/notes.txt
$ unzip docs.zip -d /tmp/out
```

gzip و پوشه‌ها

gzip برای یک فایل است و پوشه را فشرده نمی‌کند. برای فشرده‌کردن یک پوشه یا از `tar -czf` استفاده کنید یا از `zip -r` . ابزار گرافیکی **Archive Manager** در Ubuntu هم همه‌ی این قالب‌ها را باز می‌کند.

LAB 12

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

نسخه‌ی پشتیبان از یک پروژه

از پوشه‌ی یک پروژه‌ی نمونه با سه روش فشرده‌سازی پشتیبان بگیرید، حجم‌ها را مقایسه کنید، محتوا را بررسی و در جای دیگری بازیابی کنید.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. پروژه‌ی نمونه‌ی `~/project` را با چند فایل متنی و یک پوشه‌ی `logs` بسازید.

۲. سه آرشیو با gzip و bzip2 و xz بسازید که نام آنها تاریخ امروز را داشته باشد.

۳. حجم سه آرشیو را با هم مقایسه کنید.

۴. محتوای آرشیو gzip را بدون استخراج فهرست کنید.

۵. یک پشتیبان بسازید که پوشه‌ی `logs` را شامل نشود.

۶. آرشیو را در `/tmp/restore` استخراج کنید و با اصل مقایسه کنید.

۷. یک نسخه‌ی zip برای همکار Windowsای بسازید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ mkdir -p ~/project/{src,docs,logs}
$ for i in 1 2 3; do seq 1 20000 > ~/project/src/data$i.txt; done
$ echo "README" > ~/project/docs/README.md; journalctl -n 500 > ~/project/logs/app.log
$ cd ~ && D=$(date +%F)
$ tar -czf project-$D.tar.gz project
$ tar -cjf project-$D.tar.bz2 project
$ tar -cJf project-$D.tar.xz project
$ ls -lh project-*
-rw-rw-r-- 1 student student  98K Oct  4 12:41 project-2026-10-04.tar.bz2
-rw-rw-r-- 1 student student 141K Oct  4 12:41 project-2026-10-04.tar.gz
-rw-rw-r-- 1 student student  72K Oct  4 12:41 project-2026-10-04.tar.xz
```

student@ubuntu-lab: ~

```
$ tar -tzvf project-$D.tar.gz | head -4
$ tar -czf project-nologs.tar.gz --exclude='project/logs' project
$ tar -tzf project-nologs.tar.gz | grep -c logs
0
$ mkdir -p /tmp/restore && tar -xzf project-$D.tar.gz -C /tmp/restore
$ diff -r ~/project /tmp/restore/project && echo 'identical'
identical
$ zip -qr project-$D.zip project && unzip -l project-$D.zip | tail -1
```

✓ بررسی نتیجه (Verify)

ترتیب حجم باید معمولاً gzip > bzip2 > xz باشد. فرمان `diff -r` نباید هیچ تفاوتی گزارش کند و پیام identical چاپ شود. در آرشیو nologs نباید هیچ مسیری با `logs` وجود داشته باشد.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. فرق آرشیو و فشرده‌سازی چیست؟ یک ابزار برای هر کدام نام ببرید.
2. سوییچ‌های `z` و `j` و `J` در tar هر کدام کدام فشرده‌ساز را صدا می‌زنند؟
3. چطور محتوای `a.tar.xz` را بدون استخراج ببینیم؟
4. چرا tar هنگام ساخت آرشیو، `/` ابتدای مسیرها را حذف می‌کند؟
5. فرمانی بنویسید که آرشیو را در پوشه‌ی `/srv/old` استخراج کند.

Objective 3.2

وزن در آزمون: ۳

پردازش متن: ابزارهای تکمیلی

Searching and Extracting Data from Files

بیشتر فایل‌های تنظیمات و لاگ‌های لینوکس **متن ساده** هستند و ساختار ستونی دارند. مثلاً هر خط `/etc/passwd` هفت فیلد دارد که با `:` جدا شده‌اند. ابزارهای این درس مثل قطعات لگو هستند: هر کدام یک کار کوچک انجام می‌دهد و با Pipe آنها را کنار هم می‌گذارید تا گزارش دلخواه‌تان ساخته شود.

cut

برش ستون یا نویسه از هر خط

```
$ cut -d: -f1 /etc/passwd
```

sort

مرتب‌سازی خطوط (الفبایی، عددی، معکوس)

```
$ sort -n numbers.txt
```

uniq

حذف یا شمارش خطوط تکراری **پشت سر هم**

```
$ sort names | uniq -c
```

wc

شمارش خط، کلمه و نویسه

```
$ wc -l /etc/passwd
```

tr

تبدیل یا حذف نویسه‌ها

```
$ tr a-z A-Z < f
```

sed

ویرایش جریانی: جستجو و جایگزینی

```
$ sed 's/old/new/g' f
```

awk

زبان کوچک پردازش ستونی

```
$ awk '{print $1}' f
```

head / tail

ابتدا یا انتهای فایل (جلد اول)

```
$ tail -n 20 log
```

تشبیه

داده‌ی خام مثل یک گونی سیب‌زمینی است: `cut` پوست می‌کند، `sort` اندازه‌ها را مرتب می‌چیند، `uniq` تکراری‌ها را کنار می‌گذارد، `wc` می‌شمارد، `tr` و `sed` شکلشان را عوض می‌کنند و `awk` آشپزی است که همه‌ی این کارها را با هم بلد است.

برش فیلد با cut

Cutting Fields and Characters

cut

SYNTAX · ساختار کلی

```
cut -d DELIM -f FIELDS [FILE]
cut -c CHARS [FILE]
```

سوییچ‌های رایج cut

سوییچ	شکل بلند	کاربرد	مثال
-d	--delimiter	نویسه‌ی جداکننده (پیش‌فرض Tab)	cut -d: -f1 /etc/passwd
-f	--fields	شماره‌ی فیلد: 1 یا 1,3 یا 2-4	cut -d, -f2,4 a.csv
-c	--characters	بازه‌ی نویسه‌ها	cut -c1-8 file
	--complement	همه‌چیز به‌جز فیلدهای انتخابی	cut -d: -f2 --complement
-s	--only-delimited	خطوط بدون جداکننده را حذف کن	cut -s -d: -f1 f
	--output-delimiter	جداکننده‌ی خروجی متفاوت	--output-delimiter=" | "

student@ubuntu-lab: ~

```
$ cut -d: -f1 /etc/passwd | head -3  ## usernames
root
daemon
bin
$ cut -d: -f1,7 /etc/passwd | tail -2  ## user and shell
sshd:/usr/sbin/nologin
student:/bin/bash
$ date | cut -c1-10
Sun Oct  4
$ echo 'a,b,c,d' | cut -d, -f2-3 --output-delimiter=' | '
b | c
```

هشدار

cut فقط یک نویسه را به‌عنوان جداکننده می‌پذیرد و چند فاصله‌ی پشت سر هم را چند فیلد خالی می‌بیند. برای خروجی‌هایی مثل ps یا df که ستون‌ها با تعداد متغیری فاصله جدا شده‌اند، از awk استفاده کنید یا اول با tr -s ' ' فاصله‌ها را یکی کنید.

مرتب‌سازی، حذف تکرار و شمارش

sort, uniq and wc

sort

سوییچ‌های رایج sort

سوییچ	شکل بلند	کاربرد	مثال
-n	--numeric-sort	مرتب‌سازی عددی (10 بعد از 9)	sort -n nums
-r	--reverse	ترتیب معکوس	sort -rn nums
-k	--key	مرتب‌سازی بر اساس ستون	sort -k3 -n f
-t	--field-separator	جداکننده‌ی ستون	sort -t: -k3 -n /etc/passwd
-u	--unique	حذف تکراری‌ها هنگام مرتب‌سازی	sort -u names
-h	--human-numeric-sort	مقایسه‌ی 2K و 1M و 3G	du -sh * | sort -h
-f	--ignore-case	بی‌توجه به حروف کوچک و بزرگ	sort -f names

سوییچ‌های رایج uniq و wc

سوییچ	شکل بلند	کاربرد	مثال
-c	--count	uniq: تعداد تکرار هر خط	sort f | uniq -c
-d	--repeated	uniq: فقط خطوط تکراری	sort f | uniq -d
-u	--unique	uniq: فقط خطوط بدون تکرار	sort f | uniq -u
-l	--lines	wc: تعداد خطوط	wc -l file
-w	--words	wc: تعداد کلمات	wc -w file
-c	--bytes	wc: تعداد بایت	wc -c file
-m	--chars	wc: تعداد نویسه (برای فارسی)	wc -m fa.txt

student@ubuntu-lab: ~

```
$ cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn  ## which shells, how many?
     27 /usr/sbin/nologin
      2 /bin/bash
      1 /bin/sync
$ sort -t: -k3 -n /etc/passwd | tail -1 | cut -d: -f1  ## user with highest UID
nobody
$ wc /etc/passwd
  33   58 1872 /etc/passwd
```

؟ نکته‌ی آزمون

`uniq` فقط تکرارهای **پشت سر هم** را می‌بیند؛ برای همین تقریباً همیشه قبل از آن `sort` می‌آید. الگوی `sort | uniq -c | sort -rn` یعنی «پرتکرارترین‌ها اول» و در آزمون و کار واقعی بسیار رایج است. خروجی `wc` بدون سوییچ به ترتیب: خط، کلمه، بایت.

تبدیل نویسه‌ها با tr

Translating Characters

tr

SYNTAX · ساختار کلی

```
tr [OPTIONS] SET1 [SET2]      # reads only from stdin!
```

`tr` هر نویسه از مجموعه‌ی اول را با نویسه‌ی متناظر در مجموعه‌ی دوم جایگزین می‌کند. نکته‌ی مهم: `tr` نام فایل نمی‌گیرد و فقط از stdin می‌خواند؛ پس یا از Pipe استفاده کنید یا از `<` .

سوییچ‌های رایج `tr`

سوییچ	شکل بلند	کاربرد	مثال
	SET1 SET2	جایگزینی نویسه به نویسه	`tr a-z A-Z < f`
-d	--delete	حذف نویسه‌های SET1	`tr -d '\r' < win.txt`
-s	--squeeze-repeats	تکرارهای پشت سر هم را یکی کن	`tr -s ' '`
-c	--complement	همه به‌جز SET1	`tr -cd '0-9'`

student@ubuntu-lab: ~

```
$ echo 'hello linux' | tr 'a-z' 'A-Z'
HELLO LINUX
$ echo 'a::b:::c' | tr -s ':'
a:b:c
$ echo 'Phone: 0912-345-6789' | tr -cd '0-9'; echo
09123456789
$ tr ':' '\n' <<< "$PATH"  ## one PATH entry per line
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
```

نکته

فایل‌های متنی ساخته‌شده در Windows در انتهای هر خط یک نویسه‌ی اضافه‌ی `\r` دارند که اسکریپت‌ها را خراب می‌کند (خطای `bad interpreter`). با `tr -d '\r' < win.sh > linux.sh` آن را پاک کنید.

ویرایش جریانی با sed

The Stream Editor

sed

SYNTAX · ساختار کلی

```
sed [OPTIONS] 's/PATTERN/REPLACEMENT/FLAGS' [FILE]
sed [OPTIONS] 'Nd' | '/PATTERN/d' | -n 'Np' [FILE]
```

`sed` خط به خط ورودی را می‌خواند، دستوری را رویش اجرا می‌کند و نتیجه را چاپ می‌کند. پرکاربردترین دستور آن `s` (substitute) برای جستجو و جایگزینی است. به‌طور پیش‌فرض **فایل اصلی تغییر نمی‌کند** و نتیجه فقط روی صفحه می‌آید؛ برای ویرایش خود فایل از `-i` استفاده می‌کنید.

دستور / سوییچ	کار	مثال
`s/a/b/`	اولین a هر خط را با b جایگزین کن	`sed 's/cat/dog/' f`
`s/a/b/g`	همه‌ی a های هر خط (global)	`sed 's/ /_/g' f`
`s/a/b/gi`	همه، بی‌توجه به کوچک و بزرگ	`sed 's/error/ERR/gi' f`
`Nd`	حذف خط N	`sed '1d' data.csv` (حذف سرستون)
`/re/d`	حذف خطوط منطبق با الگو	`sed '/^#/d' f`
`-n 'Np'`	فقط چاپ خط N	`sed -n '5p' f`
`-n '2,4p'`	چاپ بازه‌ی خطوط	`sed -n '10,20p' f`
`-i`	ویرایش مستقیم فایل	`sed -i 's/a/b/g' f`
`-i.bak`	ویرایش با پشتیبان خودکار	`sed -i.bak 's/a/b/' f`
`-E`	Regex گسترده (ERE)	`sed -E 's/[0-9]+/N/g' f`

student@ubuntu-lab: ~

```
$ echo 'I love Windows' | sed 's/Windows/Linux/'
I love Linux
$ sed -n '1,3p' /etc/passwd | sed 's/:/ | /g'
root | x | 0 | 0 | root | /root | /bin/bash
daemon | x | 1 | 1 | daemon | /usr/sbin | /usr/sbin/nologin
bin | x | 2 | 2 | bin | /bin | /usr/sbin/nologin
$ cp /etc/ssh/sshd_config ~/sshd.test
$ sed -i.bak 's/^#Port 22/Port 2222/' ~/sshd.test
$ grep -n '^Port' ~/sshd.test; ls ~/sshd.test*
14:Port 2222
/home/student/sshd.test  /home/student/sshd.test.bak
```

⚠ هشدار

`sed -i` بدون پسوند پشتیبان، فایل را برای همیشه تغییر می‌دهد. روی فایل‌های `/etc` اول بدون `-i` اجرا کنید و خروجی را ببینید؛ وقتی مطمئن شدید، با `-i.bak` اجرا کنید تا نسخه‌ی قبلی بماند.

! **نکته**

اگر الگو یا متن جایگزین شامل `/` است (مثل مسیر فایل)، می‌توانید جداکننده را عوض کنید: `sed 's|/usr/local|/opt|g'` . هر نویسه‌ای که بعد از `s` بیاید جداکننده می‌شود.

awk

پردازش ستونی با awk

Column Processing with awk

SYNTAX · ساختار کلی

```
awk [-F SEP] 'PATTERN { ACTION }' [FILE]
```

`awk` در واقع یک زبان برنامه‌نویسی کوچک است، اما برای شروع کافی است یک چیز را بدانید: هر خط را به ستون‌ها می‌شکند و ستون‌ها را `$1` ، `$2` و... می‌نامد. `$0` کل خط است، `NF` تعداد ستون‌های خط و `NR` شماره‌ی خط. جداکننده‌ی پیش‌فرض «یک یا چند فاصله یا Tab» است، برای همین برای خروجی فرمان‌هایی مثل `df` و `ps` بهتر از `cut` کار می‌کند.

الگو	کار
`awk '{print $1}'`	ستون اول هر خط
`awk -F: '{print $1, $7}'`	ستون ۱ و ۷ با جداکننده‌ی `:`
`awk '{print $NF}'`	آخرین ستون هر خط
`awk 'NR==1'`	فقط خط اول
`awk 'NR>1 {print $5}'`	از خط دوم به بعد، ستون پنجم (حذف سرستون)
`awk -F: '$3 ⩾ 1000'`	خطوطی که ستون سوم آنها حداقل ۱۰۰۰ است
`awk '/error/ {c++} END {print c}'`	شمارش خطوط حاوی error
`awk '{s+=$1} END {print s}'`	جمع اعداد ستون اول

student@ubuntu-lab: ~

```
$ awk -F: '$3 ⩾ 1000 && $3 < 65534 {print $1, $6}' /etc/passwd  ## real users
student /home/student
$ df -h | awk 'NR>1 {print $5, $6}'  ## usage % and mount point
33% /
1% /dev/shm
6% /boot/efi
$ ps aux | awk 'NR>1 {mem+=$4} END {print "Total MEM%:", mem}'
Total MEM%: 41.7
```

؟ نکته‌ی آزمون

در سرفصل Linux Essentials از awk فقط سطح مقدماتی انتظار می‌رود: چاپ ستون با `{print $N}` و تعیین جداکننده با `-F` . بدانید `cut` و `awk` هر دو ستون جدا می‌کنند، اما `awk` فاصله‌های متغیر را بهتر مدیریت می‌کند.

LAB 13

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

تجزیه‌ی /etc/passwd و استخراج فیلدها

با cut و sort و uniq و awk و sed از فایل کاربران سیستم گزارش‌های واقعی بسازید.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. ساختار هفت فیلد `/etc/passwd` را با چاپ یک خط و شکستن آن به خطوط جدا ببینید.

۲. فهرست الفبایی نام همه‌ی کاربران را تهیه کنید و تعداد آنها را بشمارید.

۳. کاربران واقعی (UID بین ۱۰۰۰ و ۶۵۵۳۳) را همراه با پوشه‌ی خانه نمایش دهید.

۴. نشان دهید هر شِل چند بار استفاده شده است، از پرتکرار به کم‌تکرار.

۵. یک فایل CSV با ستون‌های user و uid و shell بسازید که سرستون داشته باشد.

۶. در نسخه‌ی کپی‌شده، شِل `/bin/bash` را با `/bin/zsh` جایگزین کنید و تغییر را بررسی کنید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ head -1 /etc/passwd | tr ':' '\n' | nl
     1  root
     2  x
     3  0
     4  0
     5  root
     6  /root
     7  /bin/bash
$ cut -d: -f1 /etc/passwd | sort | tee users.txt | wc -l
33
$ awk -F: '$3>=1000 && $3<65534 {print $1" -> "$6}' /etc/passwd
student -> /home/student
```

student@ubuntu-lab: ~

```
$ cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
     27 /usr/sbin/nologin
      2 /bin/bash
      1 /bin/sync
$ { echo 'user,uid,shell'; awk -F: '{print $1","$3","$7}' /etc/passwd; } > users.csv
$ head -3 users.csv
user,uid,shell
root,0,/bin/bash
daemon,1,/usr/sbin/nologin
$ cp /etc/passwd passwd.copy
$ sed -i 's|/bin/bash$|/bin/zsh|' passwd.copy
$ grep -c zsh passwd.copy; grep -c zsh /etc/passwd
2
0
```

✓ بررسی نتیجه (Verify)

فایل `users.csv` باید یک خط بیشتر از تعداد کاربران داشته باشد (سرستون). در `passwd.copy` باید دو خط با zsh ببینید و فایل اصلی `/etc/passwd` نباید هیچ تغییری کرده باشد. **هرگز** sed -i را روی خود `/etc/passwd` اجرا نکنید.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. هفت فیلد `/etc/passwd` را به ترتیب نام ببرید.
2. چرا معمولاً `uniq` بعد از `sort` می‌آید؟
3. فرمان `tr -s ' '` چه کاری انجام می‌دهد و چرا قبل از `cut` مفید است؟
4. فرق `sed 's/a/b/'` و `sed 's/a/b/g'` چیست؟
5. با awk آخرین ستون هر خط را چطور چاپ می‌کنید؟

وزن در آزمون: ۲

Objective 3.3

ویرایشگرهای nano و vim

Text Editors: nano and vim

روی سرور معمولاً محیط گرافیکی وجود ندارد و تمام فایل‌های تنظیمات را باید داخل ترمینال ویرایش کنید. Ubuntu دو ویرایشگر متنی مهم دارد: **nano** که ساده است و همه‌ی میانبرهایش پایین صفحه نوشته شده، و **vim** (نسخه‌ی پیشرفته‌ی vi) که در **همه‌ی** سیستم‌های Unix و لینوکس پیدا می‌شود و بعد از یادگیری بسیار سریع است.

ویژگی	nano	vim
یادگیری	چند دقیقه	چند روز تا چند هفته
حالت‌ها	یک حالت؛ هر کلید همان را تایپ می‌کند	چند حالت (Normal، Insert، Command)
وجود روی سرورها	بیشتر توزیع‌ها	تقریباً همه‌جا (حداقل vi)
مناسب برای	ویرایش سریع و ساده	کار حرفه‌ای و ویرایش زیاد
خروج	Ctrl+X	:q یا :wq

نکته

ویرایشگر پیش‌فرض سیستم را متغیر `EDITOR` و فرمان `sudo update-alternatives --config editor` تعیین می‌کنند. فرمان‌هایی مثل `crontab -e` و `visudo` از همین ویرایشگر استفاده می‌کنند.

nano: ویرایشگر ساده

با `nano file` باز کنید. پایین صفحه میانبرها با علامت `^` نوشته شده‌اند که یعنی کلید Ctrl؛ مثلاً `^O` یعنی Ctrl+O. `M-` یعنی کلید Alt.

کلید	کار
Ctrl+O	ذخیره (Write Out)؛ بعد Enter برای تأیید نام
Ctrl+X	خروج؛ اگر تغییر ذخیره‌نشده باشد می‌پرسد
Ctrl+W	جستجو (Where is)
Ctrl+\	جستجو و جایگزینی
Ctrl+K / Ctrl+U	بریدن خط / چسباندن
Ctrl+_	رفتن به شماره‌ی خط
Alt+U	برگرداندن آخرین تغییر (Undo)
Ctrl+G	راهنما

student@ubuntu-lab: ~

```
$ nano ~/notes.txt  ## open (or create) a file
$ sudo nano /etc/hosts  ## system files need sudo
$ nano -l /etc/ssh/sshd_config  ## -l shows line numbers
```

vim

vim: سه حالت و فرمان‌های پایه

Modes and Basic Commands in vim

مهم‌ترین نکته‌ی vim این است که **حالت‌محور** است. وقتی فایل را باز می‌کنید در حالت **Normal** هستید و کلیدها فرمان‌اند، نه متن. برای تایپ باید با `i` به حالت **Insert** بروید و برای ذخیره و خروج با `Esc` به Normal برگردید و با `:` وارد حالت **Command-line** شوید.

حالت	ورود	کار
Normal	`Esc` (حالت شروع)	حرکت، حذف، کپی، چسباندن
Insert	`i` `a` `I` `o` `A` `O`	تایپ متن؛ پایین صفحه `-- INSERT --`
Command-line	`:` از Normal	ذخیره، خروج، جستجو و جایگزینی
Visual	`v` یا `V` از Normal	انتخاب متن برای کپی یا حذف

کلید (Normal)	کار	کلید (Normal)	کار
`h` `j` `k` `l`	چپ، پایین، بالا، راست	`x`	حذف یک نویسه
`w` / `b`	کلمه‌ی بعد / قبل	`dd`	حذف (بریدن) خط
`0` / `$`	آغاز / پایان خط	`yy`	کپی خط
`gg` / `G`	اول / آخر فایل	`p` / `P`	چسباندن بعد / قبل
`:5` یا `5G`	رفتن به خط ۵	`u`	Undo
`i` / `a`	Insert قبل / بعد مکان‌نما	`Ctrl+R`	Redo
`o` / `O`	خط جدید زیر / بالا	`3dd`	حذف ۳ خط
`/word`	جستجو به جلو	`n` / `N`	نتیجه‌ی بعد / قبل

فرمان (Command-line)	کار
`:w`	ذخیره
`:q`	خروج (اگر تغییری نباشد)
`:wq` یا `:x` یا `ZZ`	ذخیره و خروج
`:q!`	خروج **بدون** ذخیره؛ دورریختن تغییرات
`:w newname`	ذخیره با نام دیگر
`:set number`	نمایش شماره‌ی خطوط
`:!cmd`	اجرای فرمان شِل بدون خروج

؟ نکته‌ی آزمون

پرسش کلاسیک: «در vim چطور بدون ذخیره خارج شویم؟» پاسخ `:q!` . همچنین: `i` به حالت Insert می‌برد، **Esc** به Normal برمی‌گرداند، `dd` خط را حذف می‌کند و `yy` کپی. ابزار آموزشی `vimtutor` را حتماً یک بار کامل اجرا کنید.

≈ تشبیه

vim مثل یک ماشین دنده‌ای است: اول باید یاد بگیرید کی کلاچ بگیرید (**Esc**) و دنده عوض کنید (`i` ، `:`)، ولی وقتی دستتان آمد، از ماشین اتوماتیک (nano) خیلی بیشتر کنترل دارید.

جستجو، جایگزینی، نشانه و کپی در vim

Search, Replace, Marks and Registers

:s///

ساختار کلی · SYNTAX

```
:[RANGE]s/PATTERN/REPLACEMENT/[FLAGS]
```

فرمان	کار
`:s/old/new/`	اولین old در خط جاری
`:s/old/new/g`	همه‌ی old ها در خط جاری
`:%s/old/new/g`	همه‌ی old ها در **کل فایل** (`%` یعنی همه‌ی خطوط)
`:%s/old/new/gc`	کل فایل، با پرسش برای هر مورد (confirm)
`:10,20s/a/b/g`	فقط خطوط ۱۰ تا ۲۰
`:%s/^#//`	حذف # از ابتدای همه‌ی خطوط
`:noh`	پاک‌کردن رنگ نتایج جستجو

نشانه‌گذاری (Bookmark) و کپی/پیست

در فایل‌های طولانی می‌توانید در یک خط **نشانه** بگذارید و بعد با یک کلید به آن برگردید. vim برای کپی و چسباندن هم «ثبات» (Register) دارد؛ هر چه با `y` کپی یا با `d` حذف کنید، در ثبات پیش‌فرض می‌رود و با `p` چسبانده می‌شود.

کلید	کار
`ma`	گذاشتن نشانه‌ی a در مکان فعلی (هر حرف a تا z)
`'a`	پرش به خطِ نشانه‌ی a
`` `a ``	پرش به مکان دقیق نشانه‌ی a
`:marks`	فهرست نشانه‌ها
`V` سپس `j` و `y`	انتخاب چند خط و کپی
`"ayy` / `"ap`	کپی خط در ثبات a / چسباندن از ثبات a
`:reg`	نمایش محتوای ثبات‌ها

! **نکته**

برای تنظیمات ماندگار، فایل `~/.vimrc` بسازید و خطوطی مثل `set number` ، `syntax on` و `set tabstop=4` را در آن بنویسید.

LAB 14

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

ویرایش فایل‌های تنظیمات نمونه

با nano و vim روی کپی فایل‌های واقعی تنظیمات کار کنید: تغییر مقدار، جستجو، جایگزینی سراسری، نشانه‌گذاری و ذخیره.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱ کپی `/etc/ssh/sshd_config` را در `~/lab14` بسازید.

۲ با nano فایل `hosts.test` بسازید، دو خط اضافه کنید و ذخیره کنید.

۳ کپی sshd_config را با vim باز کنید، شماره‌ی خطوط را روشن کنید و به خط `#Port 22` بروید.

۴ آن خط را به `Port 2222` تغییر دهید و یک خط توضیح زیرش اضافه کنید.

۵ در کل فایل `yes` را با `YES` جایگزین کنید (با تأیید هر مورد).

۶ روی خط Port نشانه‌ی `p` بگذارید، به انتهای فایل بروید و با نشانه برگردید.

۷ ذخیره و خارج شوید و تغییرات را با `diff` بررسی کنید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ mkdir -p ~/lab14 && cd ~/lab14
$ cp /etc/ssh/sshd_config sshd.conf
$ nano hosts.test  ## type 2 lines, Ctrl+O, Enter, Ctrl+X
$ cat hosts.test
192.168.56.10  web01
192.168.56.20  db01
$ vim sshd.conf
```

vim keystrokes (Normal mode)

```
:set number          " show line numbers
/#Port               " search for #Port, then Enter
0x                   " jump to line start, delete the # sign
$                    " jump to end of line (cursor on 22)
ciw2222<Esc>         " change word: 22 → 2222
o# changed in LAB14<Esc>  " open a new line below and type
mp                   " set mark p on this line
:%s/yes/YES/gc       " replace in whole file, answer y / n
G                    " go to the last line
'p                   " jump back to mark p
:wq                  " write and quit
```

student@ubuntu-lab: ~

```
$ diff /etc/ssh/sshd_config sshd.conf | head
14c14,15
< #Port 22
---
> Port 2222
> # changed in LAB14
```

✓ بررسی نتیجه (Verify)

فرمان `diff` باید تغییر خط Port، خط توضیح جدید و جایگزینی‌های yes به YES را نشان دهد. فایل اصلی `/etc/ssh/sshd_config` نباید تغییر کرده باشد (`diff` آن با فایل `.bak` یا مقایسه‌ی زمان با `ls -l` را ببینید).

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. چطور در nano ذخیره و خارج می‌شوید؟
2. سه حالت اصلی vim را نام ببرید و بگویید چطور بین آنها جابه‌جا می‌شوید.
3. فرق `:q` و `:q!` و `:wq` چیست؟
4. فرمان vim برای جایگزینی همه‌ی `http` ها با `https` در کل فایل چیست؟
5. کلیدهای `dd` ، `yy` و `p` هر کدام چه می‌کنند؟

Objective 3.4

وزن در آزمون: ۴

اسکریپت‌نویسی Bash

Turning Commands into a Script

اسکریپت Bash یک فایل متنی است که چند فرمان را پشت سر هم نگه می‌دارد تا به‌جای تایپ تکراری، با یک فرمان اجرا شوند. هر کاری که در ترمینال انجام می‌دهید، در اسکریپت هم قابل انجام است؛ به‌علاوه‌ی متغیر، شرط، حلقه و تابع. این سرفصل **پروزن‌ترین** سرفصل آزمون است (وزن ۴)، پس با دقت تمرین کنید.

Script	فایل متنی حاوی فرمان‌ها که مفسر (اینجا Bash) آن را خط به خط اجرا می‌کند.
Shebang	خط اول اسکریپت که با `#!` شروع می‌شود و مفسر را مشخص می‌کند.
Variable	نامی برای نگه‌داشتن مقدار: `NAME=value` و خواندن با `$NAME`.
Argument	مقداری که هنگام اجرا به اسکریپت می‌دهید: `./s.sh a b` ← a=`$1`.
Exit status	عدد ۰ تا ۲۵۵ که هر فرمان در پایان برمی‌گرداند؛ ۰ یعنی موفق.
Function	گروهی از فرمان‌ها با یک نام که چند بار صدا زده می‌شود.

اولین اسکریپت: از نوشتن تا اجرا

~/bin/hello.sh

```
#!/usr/bin/env bash
# hello.sh - my first script
echo "Hello, $USER!"
echo "Today is $(date +%A)"
echo "You are on $(hostname)"
```

student@ubuntu-lab: ~

```
$ mkdir -p ~/bin && nano ~/bin/hello.sh
$ chmod +x ~/bin/hello.sh  ## make it executable
$ ~/bin/hello.sh
Hello, student!
Today is Sunday
You are on ubuntu-lab
$ hello.sh  ## works if ~/bin is in PATH (re-login on Ubuntu)
$ bash ~/bin/hello.sh  ## run without x permission
```

نکته‌ی آزمون

سه روش اجرا را بدانید: `./script.sh` (نیاز به مجوز x و Shebang)، `bash script.sh` (بدون نیاز به مجوز x) و `source script.sh` یا `. script.sh` (اجرا در **همان** شِل جاری؛ متغیرها بعد از اجرا باقی می‌مانند).

Shebang، متغیرها و پارامترها

Shebang, Variables and Positional Parameters

#!

خط `#!/usr/bin/env bash` به کرنل می‌گوید این فایل را با کدام برنامه اجرا کند. `env` مسیر bash را از PATH پیدا می‌کند و قابل‌حمل‌تر از `#!/bin/bash` است. این خط حتماً باید **اولین خط** فایل باشد.

متغیر را بدون فاصله دور `=` تعریف کنید و با `$` بخوانید. خروجی یک فرمان را با `$(command)` در متغیر بریزید. همیشه متغیرها را داخل نقل‌قول دوتایی بنویسید (`"$VAR"`) تا مقادیر دارای فاصله شکسته نشوند.

متغیر ویژه	معنی
`$0`	نام اسکریپت
`$1` تا `$9` ، `${10}`	آرگومان‌های اول، دوم، …
`$#`	تعداد آرگومان‌ها
`$@`	همه‌ی آرگومان‌ها، هر کدام جدا (با `"$@"`)
`$*`	همه‌ی آرگومان‌ها به‌صورت یک رشته
`$?`	کد خروج آخرین فرمان
`$$`	PID اسکریپت جاری
`$USER` `$HOME` `$PWD`	متغیرهای محیطی رایج

greet.sh

```
#!/usr/bin/env bash
# greet.sh NAME [CITY]
name="$1"
city="${2:-Tehran}"          # default value if $2 is empty
today=$(date +%F)
echo "Script : $0"
echo "Args   : $# → $@"
echo "Hello $name from $city ($today)"
read -rp "Your favorite distro? " distro   # read input from user
echo "Nice! $distro it is."
```

student@ubuntu-lab: ~

```
$ chmod +x greet.sh && ./greet.sh Reza Mashhad
Script : ./greet.sh
Args   : 2 → Reza Mashhad
Hello Reza from Mashhad (2026-10-04)
Your favorite distro? Ubuntu
Nice! Ubuntu it is.
```

هشدار

`name = "Reza"` (با فاصله) خطاست؛ Bash فکر می‌کند `name` یک فرمان است. شکل درست: `name="Reza"` .
همچنین `'$name'` (نقل‌قول تکی) متغیر را باز نمی‌کند و همان متن `$name` را چاپ می‌کند.

شرط‌ها: if و test و case

Conditionals

if

SYNTAX · ساختار کلی

```
if [[ CONDITION ]]; then
    COMMANDS
elif [[ OTHER ]]; then
    COMMANDS
else
    COMMANDS
fi
```

`if` در واقع کد خروج یک فرمان را بررسی می‌کند: اگر ۰ بود، بخش `then` اجرا می‌شود. `[ ... ]` همان فرمان `test` است (سازگار با همه‌ی شِل‌ها)، `[[ ... ]]` نسخه‌ی پیشرفته‌ی Bash است (امن‌تر با رشته‌ها و پشتیبانی از `&&` و Regex) و `(( ... ))` برای محاسبات و مقایسه‌ی عددی است. داخل براکت‌ها **حتماً فاصله** بگذارید.

آزمون	معنی	آزمون	معنی
`-f FILE`	فایل عادی وجود دارد	`-eq` `-ne`	عدد: مساوی / نامساوی
`-d DIR`	پوشه وجود دارد	`-lt` `-le`	کوچک‌تر / کوچک‌تر مساوی
`-e PATH`	هر چیزی وجود دارد	`-gt` `-ge`	بزرگ‌تر / بزرگ‌تر مساوی
`-r` `-w` `-x`	قابل خواندن/نوشتن/اجرا	`==` `!=`	رشته: مساوی / نامساوی
`-s FILE`	فایل خالی نیست	`-z STR`	رشته خالی است
`!`	نفی شرط	`-n STR`	رشته خالی نیست
`&&` `||`	و / یا (در `[[ ]]`)	`=~`	تطبیق Regex (در `[[ ]]`)

check.sh

```
#!/usr/bin/env bash
# check.sh PATH
if [[ $# -ne 1 ]]; then
    echo "Usage: $0 PATH" >&2; exit 1
fi
if [[ -d "$1" ]]; then
    echo "$1 is a directory with $(ls "$1" | wc -l) entries"
elif [[ -f "$1" ]]; then
    echo "$1 is a file of $(stat -c %s "$1") bytes"
else
    echo "$1 does not exist"
fi
used=$(df --output=pcent / | tail -1 | tr -dc '0-9')
if (( used > 80 )); then echo "Disk almost full: $used%"; fi
case "$(id -u)" in
    0) echo "Running as root" ;;
    *) echo "Running as normal user" ;;
esac
```

نکته

میان‌بُر رایج بدون if: `cmd1 && cmd2` یعنی «اگر اولی موفق بود دومی را اجرا کن» و `cmd1 || cmd2` یعنی «اگر اولی شکست خورد دومی را». مثال: `[[ -d ~/backup ]] || mkdir ~/backup` .

for

حلقه‌ها: for، while و until

Loops

حلقه‌ها یک کار را برای چند مورد تکرار می‌کنند. `for` روی یک فهرست حرکت می‌کند (فایل‌ها، کلمات، بازه‌ی اعداد)، `while` **تا وقتی** شرط درست است تکرار می‌کند و `until` **تا وقتی** شرط درست **شود** (یعنی برعکس while).

loops.sh

```
# for over a list
for host in web01 db01 backup01; do
    echo "Checking $host"
done

# for over files (globbing)
for f in /etc/*.conf; do
    echo "$(basename "$f"): $(wc -l < "$f") lines"
done

# C-style / ranges
for (( i=1; i<=3; i++ )); do echo "Round $i"; done
for n in {1..5}; do echo -n "$n "; done; echo

# while: read a file line by line
while IFS=: read -r user _ uid _; do
    (( uid >= 1000 && uid < 65534 )) && echo "Real user: $user"
done < /etc/passwd

# until: wait for a condition
count=0
until ping -c1 -W1 8.8.8.8 &>/dev/null || (( count == 5 )); do
    echo "waiting for network..."; sleep 2; (( count++ ))
done
```

؟ نکته‌ی آزمون

در آزمون شکل کلی حلقه‌ها را بشناسید: `for VAR in LIST; do ...; done` . کلمات کلیدی پایان: `fi` برای if، `done` برای حلقه‌ها و `esac` برای case (یعنی همان کلمه برعکس). فرمان `seq 1 5` هم مثل `{1..5}` یک بازه‌ی عددی می‌سازد.

کد خروج و توابع
Exit Status and Functions

$?

هر فرمان در پایان یک **کد خروج** برمی‌گرداند: `0` یعنی موفق و هر عدد دیگر (۱ تا ۲۵۵) یعنی نوعی خطا. این عدد بلافاصله بعد از فرمان در `$?` قرار می‌گیرد و فرمان بعدی آن را بازنویسی می‌کند. اسکریپت‌های شما هم باید با `exit N` کد مناسب برگردانند.

student@ubuntu-lab: ~

```
$ ls /etc/hostname; echo $?
/etc/hostname
0
$ ls /nothing; echo $?
ls: cannot access '/nothing': No such file or directory
2
$ grep -q student /etc/passwd; echo $?  ## -q: quiet, only exit status
0
$ false; echo $?; true; echo $?
1
0
```

تابع چند فرمان را با یک نام بسته‌بندی می‌کند. تابع‌ها مثل فرمان‌های کوچک‌اند: آرگومان می‌گیرند (`$1` داخل تابع)، با `return N` یک کد وضعیت (عدد) برمی‌گردانند و برای برگرداندن **مقدار** (متن) آن را `echo` می‌کنند تا با `$( )` گرفته شود. متغیرهای داخل تابع را با `local` محلی کنید.

functions.sh

```
#!/usr/bin/env bash
set -euo pipefail          # stop on errors, unset vars, failed pipes
log() {                      # print with timestamp
    echo "[$(date +%T)] $*"
}
is_installed() {             # return status: 0 = yes
    dpkg -s "$1" &>/dev/null
}
disk_used() {                # return a VALUE via echo
    local mount="${1:-/}"
    df --output=pcent "$mount" | tail -1 | tr -dc '0-9'
}
for pkg in vim curl nginx; do
    if is_installed "$pkg"; then log "$pkg: installed"; else log "$pkg: MISSING"; fi
done
log "Root disk used: $(disk_used /)%"
exit 0
```

student@ubuntu-lab: ~

```
$ ./functions.sh
[12:58:01] vim: installed
[12:58:01] curl: installed
[12:58:01] nginx: MISSING
[12:58:01] Root disk used: 33%
```

نکته

`set -euo pipefail` در ابتدای اسکریپت‌های جدی، آنها را ایمن‌تر می‌کند: `-e` با اولین خطا متوقف می‌شود، `-u` استفاده از متغیر تعریف‌نشده را خطا می‌داند و `pipefail` شکست هر بخش از Pipe را گزارش می‌کند. برای اشکال‌زدایی هم `bash -x script.sh` هر خط را قبل از اجرا نشان می‌دهد.

LAB 15

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

اسکریپت گزارش سیستمی

اسکریپتی بنویسید که با یک فرمان، وضعیت اصلی سیستم را در قالبی خوانا گزارش دهد و در صورت پرشدن دیسک هشدار بدهد.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. فایل `~/bin/sysreport.sh` را با Shebang و توضیح بسازید.

۲. نام میزبان، توزیع، کرنل و مدت روشن بودن را چاپ کنید.

۳. درصد استفاده‌ی دیسک ریشه را در متغیر بگیرید و اگر بیشتر از حد آستانه بود هشدار دهید (آستانه با آرگومان اول، پیش‌فرض ۸۰).

۴. حافظه‌ی آزاد و تعداد کاربران واقعی را گزارش کنید.

۵. برای هر سرویس `ssh` و `cron` وضعیت را با حلقه بررسی کنید.

۶. کد خروج مناسب برگردانید (۰ سالم، ۱ هشدار).

■ راه‌حل گام‌به‌گام

~/bin/sysreport.sh

```
#!/usr/bin/env bash
# sysreport.sh [THRESHOLD] - quick health report
THRESHOLD="${1:-80}"
line() { printf '%.0s-' {1..40}; echo; }
line; echo " System report: $(hostname)  $(date '+%F %T')"; line
echo "OS      : $(grep PRETTY_NAME /etc/os-release | cut -d\" -f2)"
echo "Kernel  : $(uname -r)"
echo "Uptime  : $(uptime -p)"
used=$(df --output=pcent / | tail -1 | tr -dc '0-9')
echo "Disk /  : ${used}% used"
echo "RAM free: $(free -h | awk '/^Mem:/ {print $7}')"
echo "Users   : $(awk -F: '$3 ⩾ 1000 && $3<65534' /etc/passwd | wc -l) real user(s)"
for svc in ssh cron; do
    printf "%-8s: %s\n" "$svc" "$(systemctl is-active "$svc")"
done
line
if (( used > THRESHOLD )); then
    echo "WARNING: disk usage above ${THRESHOLD}%" >&2
    exit 1
fi
echo "All good."; exit 0
```

student@ubuntu-lab: ~

```
$ chmod +x ~/bin/sysreport.sh && ~/bin/sysreport.sh
----------------------------------------
 System report: ubuntu-lab  2026-10-04 13:05:12
----------------------------------------
OS      : Ubuntu 24.04.1 LTS
Kernel  : 6.8.0-45-generic
Uptime  : up 1 hour, 20 minutes
Disk /  : 33% used
RAM free: 2.4Gi
Users   : 1 real user(s)
ssh     : active
cron    : active
----------------------------------------
All good.
$ ~/bin/sysreport.sh 20; echo "exit=$?"  ## low threshold to test the warning
WARNING: disk usage above 20%
exit=1
```

✓ بررسی نتیجه (Verify)

اسکریپت بدون آرگومان باید `All good.` و کد خروج ۰ بدهد و با آستانه‌ی پایین (مثلاً ۲۰) پیام هشدار و کد ۱. اگر خطای `Permission denied` گرفتید، مجوز x را فراموش کرده‌اید؛ اگر `bad interpreter` دیدید، خط Shebang را بررسی کنید.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. خط Shebang چیست و چرا باید خط اول باشد؟
2. عبارت `${1:-80}` چه معنایی دارد؟
3. چرا پیام هشدار را با `>&2` چاپ کردیم؟
4. فرق `[ ]` و `[[ ]]` و `(( ))` چیست؟
5. چطور یک اسکریپت را خط به خط برای اشکال‌زدایی اجرا کنیم؟

LAB
16

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

اسکریپت پشتیبان‌گیری با تاریخ

یک اسکریپت پشتیبان‌گیری واقعی بنویسید که پوشه‌ها را با نام تاریخ‌دار آرشیو کند، فقط ۵ نسخه‌ی آخر را نگه دارد و همه‌چیز را در لاگ ثبت کند.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. اسکریپت `~/bin/backup.sh` بسازید که یک یا چند پوشه را به‌عنوان آرگومان بگیرد.

۲. اگر آرگومانی داده نشد، پیام راهنما بدهد و با کد ۱ خارج شود.

۳. برای هر پوشه بررسی کند وجود دارد؛ اگر نه، خطا را در لاگ بنویسد و ادامه دهد.

۴. آرشیو را با نام `NAME-YYYY-MM-DD_HHMM.tar.gz` در `~/backups` بسازد.

۵. فقط ۵ پشتیبان آخر هر پوشه را نگه دارد و بقیه را پاک کند.

۶. خلاصه‌ی کار (تعداد موفق و ناموفق) را چاپ کند.

■ راه‌حل گام‌به‌گام

~/bin/backup.sh

```
#!/usr/bin/env bash
# backup.sh DIR... - dated tar.gz backups with rotation
set -uo pipefail
DEST="$HOME/backups"; KEEP=5
LOG="$DEST/backup.log"
mkdir -p "$DEST"
log() { echo "[$(date '+%F %T')] $*" | tee -a "$LOG"; }

if (( $# == 0 )); then
    echo "Usage: $0 DIR [DIR...]" >&2; exit 1
fi
ok=0; fail=0
for src in "$@"; do
    if [[ ! -d "$src" ]]; then
        log "ERROR: $src is not a directory"; (( fail++ )); continue
    fi
    name=$(basename "$src")
    file="$DEST/${name}-$(date +%F_%H%M).tar.gz"
    if tar -czf "$file" -C "$(dirname "$src")" "$name" 2>>"$LOG"; then
        log "OK: $src → $file ($(du -h "$file" | cut -f1))"; (( ok++ ))
    else
        log "ERROR: tar failed for $src"; (( fail++ ))
    fi
    # rotation: keep newest $KEEP files of this name
    ls -1t "$DEST/${name}"-*.tar.gz | tail -n +$((KEEP+1)) | while read -r old; do
        rm -f "$old" && log "Rotated: $old"
    done
done
log "Done: $ok ok, $fail failed"
(( fail == 0 ))     # exit status 0 only if everything worked
```

student@ubuntu-lab: ~

```
$ chmod +x ~/bin/backup.sh
$ ~/bin/backup.sh; echo $?
Usage: /home/student/bin/backup.sh DIR [DIR...]
1
$ ~/bin/backup.sh ~/project ~/Documents /nope
[2026-10-04 13:20:01] OK: /home/student/project → /home/student/backups/
project-2026-10-04_1320.tar.gz (140K)
[2026-10-04 13:20:01] OK: /home/student/Documents → /home/student/backups/
Documents-2026-10-04_1320.tar.gz (4.0K)
[2026-10-04 13:20:01] ERROR: /nope is not a directory
[2026-10-04 13:20:01] Done: 2 ok, 1 failed
```

۱ زمان‌بندی با cron

برای اجرای خودکار، `crontab -e` را بزنید و این خط را اضافه کنید: `0 2 * * * $HOME/bin/backup.sh $HOME/project` . پنج فیلد به ترتیب: دقیقه، ساعت، روز ماه، ماه و روز هفته.

✓ **بررسی نتیجه (Verify)**

در `~/backups` باید فایل‌های تاریخ‌دار و `backup.log` ببینید. اسکریپت را چند بار با فاصله‌ی یک دقیقه اجرا کنید؛ بعد از اجرای ششم، از هر پوشه فقط ۵ فایل باقی بماند (`ls ~/backups | grep -c project`). کد خروج با وجود `/nope` باید ۱ باشد.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. فرمان `date +%F_%H%M` چه خروجی‌ای می‌دهد؟
2. چرا از `"$@"` (با نقل‌قول) در حلقه استفاده کردیم؟
3. فرمان `continue` در حلقه چه کار می‌کند؟
4. منطق چرخش (Rotation) با `ls -1t | tail -n +6` چطور کار می‌کند؟
5. چطور این اسکریپت را هر شب ساعت ۲ خودکار اجرا کنیم؟ (راهنما: `crontab -e`)

EXAM 010-160 · TOPIC 4

4

سیستم‌عامل لینوکس و شبکه

THE LINUX OPERATING SYSTEM

حالا که خط فرمان را بلدید، وقت آن است که زیر کاپوت را ببینیم: چه سیستم‌عاملی انتخاب کنیم، سخت‌افزار و دیسک را چطور بشناسیم، فایل‌ها کجا ذخیره می‌شوند، فرایندها و سرویس‌ها و لاگ‌ها چطور کار می‌کنند و در پایان کارت شبکه را با netplan روی یک IP ثابت تنظیم کنیم.

وزن در آزمون: ۸ سؤال از ۴۰

4.1	انتخاب سیستم‌عامل
4.2	سخت‌افزار و دیسک
4.3	FHS و محل ذخیره‌ی داده‌ها
4.4	فرایندها، سرویس‌ها و لاگ‌ها
4.5	شبکه و IP
4.6	تنظیم IP ثابت با netplan

وزن در آزمون: ۱

Objective 4.1

انتخاب سیستم‌عامل

Choosing an Operating System

انتخاب سیستم‌عامل یک تصمیم فنی و اقتصادی است: چه نرم‌افزاری باید اجرا شود، چه کسی پشتیبانی می‌کند، تا کی به‌روزرسانی امنیتی می‌گیرد و هزینه‌ی مجوز چقدر است. در دنیای لینوکس، مهم‌ترین تفاوت توزیع‌ها در **چرخه‌ی انتشار** است.

معیار	LTS / Stable	Rolling Release
تعریف	نسخه‌های ثابت با دوره‌ی پشتیبانی مشخص	بدون نسخه؛ بسته‌ها مدام به‌روز می‌شوند
نمونه‌ها	Ubuntu LTS، Debian Stable، RHEL، Rocky	Arch، openSUSE Tumbleweed، Gentoo
پایداری	بسیار بالا؛ تغییرات کم و آزموده	کمتر؛ نسخه‌های جدید زود می‌رسند
نسخه‌ی نرم‌افزارها	قدیمی‌تر اما امن	همیشه آخرین نسخه
مناسب برای	سرور، سازمان، تولید (Production)	دسکتاپ توسعه‌دهنده، علاقه‌مندان

دوره‌ی عمر Ubuntu	توضیح
LTS (مثل 24.04)	هر دو سال، آوریل سال‌های زوج؛ **۵ سال** پشتیبانی استاندارد
Interim (مثل 24.10)	هر شش ماه؛ فقط **۹ ماه** پشتیبانی؛ برای آزمودن امکانات تازه
ESM با Ubuntu Pro	پشتیبانی امنیتی گسترده تا **۱۰ سال** (و با افزونه‌ی Legacy تا ۱۲ سال)
EOL	End of Life؛ پایان پشتیبانی. سرور EOL دیگر وصله‌ی امنیتی نمی‌گیرد

Ubuntu Pro و ESM

اشتراک **Ubuntu Pro** برای استفاده‌ی شخصی تا ۵ ماشین رایگان است. با آن پوشش امنیتی ESM برای مخزن‌های main و universe، وصله‌ی زنده‌ی کرنل (Livepatch) و ابزارهای انطباق امنیتی فعال می‌شود. فرمان‌ها: `pro status` ، `sudo pro attach TOKEN` .

نکته‌ی آزمون

در آزمون ممکن است بپرسند کدام توزیع برای سرور سازمانی مناسب‌تر است (توزیع با پشتیبانی بلند مثل RHEL یا Ubuntu LTS)، یا «Rolling Release» یعنی چه. همچنین بدانید سیستم‌عامل‌های دیگر مثل Windows Server و macOS هم انتخاب‌های رایج‌اند و انتخاب به نیاز نرم‌افزاری بستگی دارد.

شناسایی ماشین با hostnamectl

System Identity

hostnamectl

SYNTAX · ساختار کلی

```
hostnamectl [status | set-hostname NAME | hostname]
```

`hostnamectl` بخشی از systemd است و در یک نگاه نام ماشین، نوع (VM یا فیزیکی)، سیستم‌عامل، کرنل، معماری، سازنده‌ی سخت‌افزار و تاریخ پایان پشتیبانی را نشان می‌دهد. با آن می‌توانید نام ماشین را هم به‌طور دائمی تغییر دهید.

student@ubuntu-lab: ~

```
$ hostnamectl
 Static hostname: ubuntu-lab
       Icon name: computer-vm
         Chassis: vm
      Machine ID: 4c1f2b0f6e8e4c42a9f2c6f1b7a3d901
  Virtualization: vmware
Operating System: Ubuntu 24.04.1 LTS
          Kernel: Linux 6.8.0-45-generic
    Architecture: x86-64
 Hardware Vendor: VMware, Inc.
  Hardware Model: VMware Virtual Platform
Firmware Version: 6.00
$ sudo hostnamectl set-hostname ubuntu-lab2  ## permanent rename
$ hostname
ubuntu-lab2
$ pro status | head -5
```

هشدار

بعد از تغییر hostname، فایل `/etc/hosts` را هم به‌روز کنید (خط `127.0.1.1 ubuntu-lab`)؛ وگرنه sudo پیغام `unable to resolve host` می‌دهد. در این دوره نام را به `ubuntu-lab` برگردانید تا خروجی‌ها با جزوه یکسان بماند.

LAB 17

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

شناسنامه‌ی سیستم و دوره‌ی پشتیبانی

برای ماشین آزمایشگاه یک «شناسنامه» تهیه کنید: نوع ماشین، نسخه، دوره‌ی پشتیبانی و وضعیت Ubuntu Pro.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. خروجی کامل `hostnamectl` را در فایل `~/lab17/identity.txt` ذخیره کنید.

۲. فقط خطوط Operating System و Kernel و Virtualization را بیرون بکشید.

۳. نام ماشین را موقتاً به `lab17-test` تغییر دهید، بررسی کنید و برگردانید.

۴. وضعیت Ubuntu Pro و سرویس‌های ESM را ببینید.

۵. با `ubuntu-distro-info` تاریخ پایان پشتیبانی نسخه‌ی فعلی را پیدا کنید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ mkdir -p ~/lab17 && hostnamectl > ~/lab17/identity.txt
$ grep -E 'Operating|Kernel|Virtualization' ~/lab17/identity.txt
  Virtualization: vmware
Operating System: Ubuntu 24.04.1 LTS
          Kernel: Linux 6.8.0-45-generic
$ sudo hostnamectl set-hostname lab17-test && hostnamectl hostname
lab17-test
$ sudo hostnamectl set-hostname ubuntu-lab
$ pro status 2>/dev/null | head -4
SERVICE          ENTITLED  STATUS       DESCRIPTION
esm-apps         yes       disabled     Expanded Security Maintenance for Applications
esm-infra        yes       disabled     Expanded Security Maintenance for Infrastructure
$ sudo apt install -y distro-info && ubuntu-distro-info --series=noble -y=eol
1666  ## days until end of standard support
```

✓ بررسی نتیجه (Verify)

در پایان باید `hostname` دوباره `ubuntu-lab` را نشان دهد. خروجی `ubuntu-distro-info` عدد روزهای باقی‌مانده تا پایان پشتیبانی (آوریل ۲۰۲۹) را نشان می‌دهد؛ عدد شما به تاریخ امروز بستگی دارد.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. فرق LTS و Rolling Release چیست؟ از هر کدام یک توزیع نام ببرید.
2. Ubuntu 24.04 LTS بدون Ubuntu Pro چند سال پشتیبانی دارد؟
3. منظور از EOL چیست و چرا سرور EOL خطرناک است؟
4. کدام فرمان نام ماشین را برای همیشه تغییر می‌دهد؟
5. چرا برای سرور تولیدی معمولاً Rolling Release انتخاب نمی‌شود؟

وزن در آزمون: ۲

Objective 4.2

شناخت سخت‌افزار و دیسک

Understanding Computer Hardware

مدیر سیستم باید بتواند بدون بازکردن کیس بفهمد ماشین چه پردازنده‌ای، چقدر حافظه، چه دیسک‌هایی و چه دستگاه‌هایی دارد. لینوکس همه‌ی این اطلاعات را از کرنل (فایل‌های `/proc` و `/sys`) می‌خواند و ابزارهای زیر آنها را خوانا نمایش می‌دهند.

CPU	پردازنده؛ هسته‌ها (Core) و رشته‌ها (Thread). معماری رایج: x86_64 و ARM (aarch64).
RAM	حافظه‌ی اصلی موقت؛ با خاموش‌شدن پاک می‌شود.
Swap	فضایی روی دیسک که وقتی RAM پر شود، به‌جای حافظه استفاده می‌شود؛ بسیار کندتر.
Disk / Partition	دیسک فیزیکی (sda، nvme0n1) به پارتیشن‌ها (sda1، sda2) تقسیم می‌شود.
Filesystem	قالب ذخیره‌ی فایل روی پارتیشن؛ مثل ext4، xfs، btrfs، vfat.
Mount point	پوشه‌ای که یک فایل‌سیستم به آن وصل می‌شود؛ مثل `/` یا `/boot/efi`.
Driver / Module	کد کرنل برای کار با یک دستگاه؛ با `lsmod` فهرست می‌شود.

≈ تشبیه

RAM میز کار شماست و دیسک کمد بایگانی. هر چه میز بزرگ‌تر باشد پرونده‌های بیشتری را هم‌زمان باز نگه می‌دارید. Swap مثل گذاشتن پرونده‌ها روی زمین است وقتی میز پر شده؛ کار می‌کند ولی هر بار باید خم شوید و دنبالشان بگردید.

lscpu

مشخصات پردازنده: مدل، هسته، معماری

```
$ lscpu | head
```

free -h

RAM و Swap به‌صورت خوانا

```
$ free -h
```

lsblk

دیسک‌ها و پارتیشن‌ها به شکل درخت

```
$ lsblk -f
```

df -h

فضای آزاد هر فایل‌سیستم mountشده

```
$ df -hT
```

du -sh

اندازه‌ی یک پوشه

```
$ du -sh /var/log
```

lspci

دستگاه‌های PCI (کارت شبکه، گرافیک)

```
$ lspci | grep -i net
```

lsusb

دستگاه‌های USB

```
$ lsusb
```

lsmod

ماژول‌های (درایورهای) بارگذاری‌شده

```
$ lsmod | head
```

پردازنده و حافظه: lscpu و free

CPU and Memory

lscpu

```
student@ubuntu-lab: ~
$ lscpu | grep -E '^(Architecture|CPU\(s\)|Model name|Thread|Core|Hypervisor)'
Architecture:          x86_64
CPU(s):                2
Model name:            Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz
Thread(s) per core:    1
Core(s) per socket:    1
Hypervisor vendor:     VMware
$ nproc  ## number of CPUs only
2
$ grep -c ^processor /proc/cpuinfo  ## the raw source
2
```

سوییچ‌های رایج free

سوییچ	شکل بلند	کاربرد	مثال
-h	--human	واحد خوانا (Mi، Gi)	free -h
-m	--mebi	نمایش به مگابایت	free -m
-s N	--seconds	تکرار هر N ثانیه	free -h -s 2
-t	--total	افزودن سطر جمع	free -ht

```
student@ubuntu-lab: ~
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       1.2Gi       1.4Gi        38Mi       1.5Gi       2.4Gi
Swap:          3.8Gi          0B       3.8Gi
$ swapon --show
NAME      TYPE SIZE USED PRIO
/swap.img file 3.8G   0B   -2
```

نکته

ستون free را با حافظه‌ی واقعاً آزاد اشتباه نگیرید. لینوکس حافظه‌ی بیکار را برای کش دیسک (buff/cache) استفاده می‌کند و هر وقت برنامه‌ای لازم داشت آزادش می‌کند. عدد مهم ستون **available** است.

lsblk

دیسک و فضا: lsblk، df و du

Disks, Partitions and Space

در لینوکس هر دیسک یک فایل دستگاه در `/dev` دارد: دیسک‌های SATA/SCSI و دیسک مجازی VMware با نام `sda`، `sdb` … و دیسک‌های NVMe با نام `nvme0n1`. پارتیشن‌ها با یک عدد اضافه نام‌گذاری می‌شوند: `sda1`، `sda2` یا `nvme0n1p1`.

student@ubuntu-lab: ~

```
$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   30G  0 disk
├─sda1   8:1    0    1M  0 part
└─sda2   8:2    0   30G  0 part /
sr0     11:0    1  5.8G  0 rom
$ lsblk -f /dev/sda2  ## filesystem type and UUID
NAME FSTYPE FSVER LABEL UUID                                 FSAVAIL FSUSE% MOUNTPOINTS
sda2 ext4   1.0         9b1c...e7a2                            19G    33% /
```

سوییچ‌های رایج `df` و `du`

سوییچ	شکل بلند	کاربرد	مثال
`-h`	`--human-readable`	df/du: واحد خوانا	`df -h`
`-T`	`--print-type`	df: نوع فایل‌سیستم	`df -hT`
`-i`	`--inodes`	df: inode های آزاد	`df -i /`
`-s`	`--summarize`	du: فقط جمع کل	`du -sh ~`
`-a`	`--all`	du: همه‌ی فایل‌ها	`du -ah dir`
`-d N`	`--max-depth`	du: تا عمق N	`du -h -d1 /var`
`-x`	`--one-file-system`	du: نرفتن به فایل‌سیستم دیگر	`sudo du -xh -d1 /`

student@ubuntu-lab: ~

```
$ df -hT -x tmpfs
Filesystem     Type  Size  Used Avail Use% Mounted on
/dev/sda2      ext4   30G  9.1G   19G  33% /
$ sudo du -h -d1 /var 2>/dev/null | sort -h | tail -4
148M    /var/lib
412M    /var/cache
61M     /var/log
640M    /var
```

؟ نکته‌ی آزمون

فرق `df` و `du` پرسش رایج است: `df` فضای **فایل‌سیستم‌ها** را از دید کل دیسک گزارش می‌کند و `du` حجم **فایل‌ها و پوشه‌ها** را می‌شمارد. همچنین بدانید `/dev/sda` کل دیسک است و `/dev/sda1` اولین پارتیشن آن.

lspci، lsusb، lsmod: دستگاه‌ها و درایورها

Devices and Drivers

lspci

student@ubuntu-lab: ~

```
$ lspci | grep -iE 'ethernet|vga|sata'
00:0f.0 VGA compatible controller: VMware SVGA II Adapter
02:01.0 Ethernet controller: Intel Corporation 82545EM Gigabit Ethernet Controller
02:05.0 SATA controller: VMware SATA AHCI controller
$ lspci -k -s 02:01.0  ## which driver is used?
        Kernel driver in use: e1000
$ lsusb
Bus 002 Device 002: ID 0e0f:0003 VMware, Inc. Virtual Mouse
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
$ lsmod | grep -E '^(e1000|vmw)'
e1000                  180224  0
vmw_balloon             28672  0
vmwgfx                 434176  2
```

VM

VMware Workstation

در VMware، کارت شبکه‌ی مجازی معمولاً «Intel 82545EM» با درایور `e1000` یا «VMXNET3» با درایور `vmxnet3` است. نام کارت در لینوکس (مثل `ens33`) از شماره‌ی شکاف PCI ساخته می‌شود؛ برای همین روی سیستم شما ممکن است متفاوت باشد.

I

درایورها

بیشتر درایورها در لینوکس **ماژول کرنل** هستند و هنگام شناسایی دستگاه به‌طور خودکار بار می‌شوند. درایورهای اختصاصی (مثلاً کارت گرافیک NVIDIA) را در Ubuntu با `ubuntu-drivers devices` و `sudo ubuntu-drivers install` نصب کنید.

LAB 18

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

مشاهده‌ی تمام مشخصات سخت‌افزار VM

برای ماشین مجازی یک برگه‌ی مشخصات کامل تهیه کنید و ببینید تنظیمات VMware چطور در لینوکس دیده می‌شوند.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. معماری، تعداد CPU و مدل پردازنده را پیدا کنید.

۲. حافظه‌ی کل، قابل‌استفاده و Swap را گزارش کنید.

۳. درخت دیسک‌ها را با نوع فایل‌سیستم ببینید.

۴. فضای آزاد ریشه و ۳ پوشه‌ی بزرگ `/usr` را پیدا کنید.

۵. کارت‌های شبکه و درایور آن‌ها را با lspci شناسایی کنید.

۶. همه را با یک فرمان در `~/lab18/hw.txt` جمع کنید.

۷. در VMware حافظه‌ی VM را به ۴۵۰۰ مگابایت تغییر دهید و تأثیرش را ببینید (اختیاری).

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ mkdir -p ~/lab18 && cd ~/lab18
$ lscpu | grep -E '^(Architecture|CPU\(s\)|Model name)'
$ free -h | awk 'NR<=3'
$ lsblk -f
$ df -h /
$ du -sh /usr/* 2>/dev/null | sort -rh | head -3
2.9G    /usr/lib
1.1G    /usr/share
214M    /usr/bin
$ lspci -k | grep -A2 -i ethernet
02:01.0 Ethernet controller: Intel Corporation 82545EM Gigabit Ethernet Controller
        Subsystem: VMware PRO/1000 MT Single Port Adapter
        Kernel driver in use: e1000
```

student@ubuntu-lab: ~

```
$ { echo '== CPU =='; lscpu | head -15; echo '== RAM =='; free -h
>   echo '== DISK =='; lsblk -f; df -hT -x tmpfs
>   echo '== PCI =='; lspci; echo '== USB =='; lsusb; } > hw.txt 2>&1
$ wc -l hw.txt
74 hw.txt
```

✓ بررسی نتیجه (Verify)

تعداد CPU باید با تنظیم (۲) VMware و حافظه‌ی کل حدود ۳٫۸ گیگابایت باشد (کمی کمتر از ۴ گیگابایت، چون کرنل بخشی را رزرو می‌کند). باید دو کارت Ethernet در lspci ببینید؛ یکی برای NAT و یکی برای Host-Only.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. کدام فرمان معماری پردازنده را نشان می‌دهد؟ دو روش بنویسید.
2. چرا ستون `available` در free مهم‌تر از `free` است؟
3. فرق `/dev/sda` و `/dev/sda2` چیست؟
4. فرق `df -h` و `du -sh` را با یک مثال توضیح دهید.
5. کدام فرمان درایور در حال استفاده‌ی یک کارت PCI را نشان می‌دهد؟

وزن در آزمون: ۳

Objective 4.3

FHS و سازمان سیستم

Where Data is Stored: The Filesystem Hierarchy

در Windows برنامه‌ها معمولاً همه‌ی فایل‌هایشان را در یک پوشه در `Program Files` می‌گذارند. لینوکس برعکس عمل می‌کند: فایل‌ها بر اساس **نوع** تقسیم می‌شوند؛ همه‌ی فایل‌های اجرایی در یک جا، همه‌ی تنظیمات در جای دیگر و همه‌ی لاگ‌ها در جای سوم. این نظم را استاندارد **FHS** (Filesystem Hierarchy Standard) تعریف می‌کند و تقریباً همه‌ی توزیع‌ها از آن پیروی می‌کنند.

مسیر	محتوا	مثال
`/`	ریشه؛ بالای درخت. همه‌چیز زیر آن است	—
`/etc`	فایل‌های **تنظیمات** سیستم و سرویس‌ها؛ متنی	`/etc/hostname` ، `/etc/ssh/`
`/home`	پوشه‌ی خانه‌ی کاربران عادی	`/home/student`
`/root`	خانه‌ی کاربر root (نه زیر `/home`)	`/root/.bashrc`
`/usr/bin`	برنامه‌های کاربران (اکثر فرمان‌ها)	`/usr/bin/ls`
`/usr/sbin`	برنامه‌های مدیریتی سیستم	`/usr/sbin/useradd`
`/usr/lib`	کتابخانه‌های مشترک برنامه‌ها	`/usr/lib/x86_64-linux-gnu/`
`/usr/share`	داده‌های مستقل از معماری؛ مستندات و man	`/usr/share/doc/`
`/usr/local`	نرم‌افزارهایی که خودتان دستی نصب می‌کنید	`/usr/local/bin/`
`/opt`	نرم‌افزارهای اختیاری شخص ثالث	`/opt/google/`
`/var`	داده‌های **متغیر**: لاگ، کش، صف چاپ، وب	`/var/log` ، `/var/www`
`/var/log`	لاگ‌های سیستم و سرویس‌ها	`/var/log/syslog`
`/tmp`	فایل‌های موقت؛ با هر راه‌اندازی پاک می‌شود	—
`/run`	داده‌های زمان اجرا (PID، سوکت) در RAM	`/run/sshd.pid`
`/boot`	کرنل، initrd و GRUB	`/boot/vmlinuz-6.8.0-45-generic`
`/dev`	فایل‌های دستگاه	`/dev/sda` ، `/dev/null`
`/proc` `/sys`	فایل‌سیستم‌های **مجازی** کرنل؛ روی دیسک نیستند	`/proc/cpuinfo`
`/media` `/mnt`	محل mount فلش و دیسک (خودکار / دستی)	`/media/student/USB`

UsrMerge

در Ubuntu جدید، پوشه‌های `/bin` ، `/sbin` و `/lib` فقط پیوند نمادی به `/usr/bin` ، `/usr/sbin` و `/usr/lib` هستند. به این طرح **UsrMerge** می‌گویند. پس `/bin/ls` و `/usr/bin/ls` یک فایل‌اند.

؟ نکته‌ی آزمون

پرسش‌های رایج: تنظیمات کجاست؟ (`/etc`). لاگ‌ها؟ (`/var/log`). خانه‌ی root؟ (`/root`). اطلاعات زنده‌ی فرایندها و سخت‌افزار؟ (`/proc`). کرنل و بوت‌لودر؟ (`/boot`). فایل‌هایی که بعد از راه‌اندازی مجدد پاک می‌شوند؟ (`/tmp`).

/proc

فایل‌سیستم‌های مجازی: /proc و /sys

Virtual Filesystems

/proc و /sys روی دیسک وجود ندارند؛ کرنل آنها را لحظه‌ای در حافظه می‌سازد. هر بار که فایلی از آنها را می‌خوانید، کرنل اطلاعات تازه را تولید می‌کند. برای هر فرایند در حال اجرا یک پوشه با شماره‌ی PID آن در /proc وجود دارد.

فایل	محتوا
/proc/cpuinfo	مشخصات هر هسته‌ی پردازنده
/proc/meminfo	جزئیات حافظه (منبع free)
/proc/version	نسخه‌ی کرنل
/proc/uptime	ثانیه‌های روشن‌بودن
/proc/loadavg	بار سیستم (منبع uptime)
/proc/PID/	اطلاعات یک فرایند: cmdline ، status
/proc/sys/	تنظیمات قابل‌تغییر کرنل (sysctl)
/sys/class/net/	کارت‌های شبکه و وضعیت آنها

student@ubuntu-lab: ~

```
$ head -3 /proc/meminfo
MemTotal:        3996352 kB
MemFree:         1468212 kB
MemAvailable:    2516820 kB
$ cat /proc/loadavg
0.08 0.12 0.09 1/412 2754
$ cat /proc/$$/cmdline; echo  ## $$ = PID of this shell
bash
$ ls /sys/class/net
ens33  ens37  lo
$ cat /sys/class/net/ens33/address
00:0c:29:7a:1b:2c
$ cat /proc/sys/net/ipv4/ip_forward
0
```

تشبیه

/proc مثل داشبورد خودرو است: سرعت‌سنج و دورسنج قطعه‌ی واقعی موتور نیستند، فقط وضعیت لحظه‌ای موتور را نشان می‌دهند. اگر فایل /proc/meminfo را کپی کنید، فقط یک عکس از داشبورد گرفته‌اید.

LAB 19

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

پیمایش FHS و فهم ساختار

در درخت استاندارد لینوکس گشت بزنید و بفهمید هر فایل مهم کجاست و چرا آنجاست.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. سطح اول درخت `/` را با `ls -l` ببینید و پیوندهای UsrMerge را تشخیص دهید.

۲. محل فایل اجرایی `ls`، `useradd` و `sshd` را پیدا کنید.

۳. فایل تنظیمات SSH و لاگ احراز هویت را پیدا کنید.

۴. نسخه‌ی کرنل را از `/boot` و `/proc` بخوانید.

۵. نشان دهید `/proc` روی دیسک حجمی ندارد.

۶. فایلی در `/tmp` بسازید، سیستم را ری‌استارت کنید و ببینید هنوز هست یا نه.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ ls -l / | grep -E '^l'
lrwxrwxrwx   1 root root     7 Apr 22 13:08 bin → usr/bin
lrwxrwxrwx   1 root root     7 Apr 22 13:08 lib → usr/lib
lrwxrwxrwx   1 root root     8 Apr 22 13:08 sbin → usr/sbin
$ which ls useradd; ls -l /usr/sbin/sshd
/usr/bin/ls
/usr/sbin/useradd
-rwxr-xr-x 1 root root 921416 Aug  9 18:25 /usr/sbin/sshd
$ ls /etc/ssh/sshd_config /var/log/auth.log
$ ls /boot/vmlinuz-*; uname -r
/boot/vmlinuz-6.8.0-45-generic
6.8.0-45-generic
$ df -h /proc; du -sh /proc 2>/dev/null
Filesystem      Size  Used Avail Use% Mounted on
proc               0     0     0    - /proc
0       /proc
$ echo test > /tmp/lab19.txt && sudo reboot
$ ls /tmp/lab19.txt  ## after reboot
ls: cannot access '/tmp/lab19.txt': No such file or directory
```

✓ بررسی نتیجه (Verify)

باید پیوندهای `bin → usr/bin` و مانند آن را ببینید. اندازه‌ی `/proc` صفر است چون مجازی است. فایل `/tmp/lab19.txt` بعد از راه‌اندازی مجدد حذف شده است؛ اگر هنوز بود، روی سیستم شما `/tmp` پاک‌سازی را فقط بر اساس زمان انجام می‌دهد (systemd-tmpfiles).

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. فایل‌های تنظیمات سیستم در کدام پوشه هستند؟
2. فرق `/usr/bin` و `/usr/sbin` چیست؟
3. چرا لاگ‌ها در `/var` هستند و نه در `/usr`؟
4. دو فایل‌سیستم مجازی کرنل را نام ببرید.
5. نرم‌افزاری که خودتان از کد منبع کامپایل می‌کنید معمولاً کجا نصب می‌شود؟

وزن در آزمون: ۳

Objective 4.4

فرایندها، سرویس‌ها و لاگ‌ها

Processes, Services and Logs

هر برنامه‌ای که اجرا می‌شود، یک **فرایند** (Process) است و یک شماره‌ی یکتا به نام **PID** دارد. اولین فرایندی که کرنل اجرا می‌کند `systemd` با PID ۱ است و بقیه‌ی فرایندها فرزندان آن هستند. فرایندهایی که در پس‌زمینه همیشه اجرا می‌مانند (مثل سرور SSH) **سرویس** یا **Daemon** نام دارند و معمولاً نامشان با `d` تمام می‌شود: `sshd` ، `cron` ، `systemd-journald` .

PID / PPID	شماره‌ی فرایند و شماره‌ی فرایند والد.
Daemon	فرایند پس‌زمینه بدون ترمینال؛ سرویس‌دهنده.
systemd	مدیر سیستم و سرویس‌ها در Ubuntu؛ PID ۱. واحدهای آن Unit نام دارند (`ssh.service`).
Signal	پیامی به فرایند: `SIGTERM` (۱۵، خاتمه‌ی مؤدبانه)، `SIGKILL` (۹، کشتن اجباری)، `SIGHUP` (۱، بارگذاری مجدد).
Load average	میانگین تعداد فرایندهای منتظر CPU در ۱، ۵ و ۱۵ دقیقه‌ی اخیر.
Journal	پایگاه لاگ systemd؛ با `journalctl` خوانده می‌شود.

ps

عکس فوری فرایندها: ps و kill

Process Snapshot

سوییچ‌های رایج ps

سوییچ	شکل بلند	کاربرد	مثال
aux		همه‌ی فرایندها، همه‌ی کاربران (سبک BSD)	ps aux
-ef		همه‌ی فرایندها با PPID (سبک Unix)	ps -ef
-u	--user	فرایندهای یک کاربر	ps -u student
-C		بر اساس نام فرمان	ps -C sshd
-o	--format	انتخاب ستون‌ها	ps -o pid,cmd
	--sort	مرتب‌سازی	ps aux --sort=-%mem
--forest		نمایش درختی والد و فرزند	ps -ef --forest

student@ubuntu-lab: ~

```
$ ps
    PID TTY          TIME CMD
   2311 pts/0    00:00:00 bash
   2790 pts/0    00:00:00 ps
$ ps aux --sort=-%mem | head -4
USER     PID %CPU %MEM    VSZ   RSS TTY  STAT START   TIME COMMAND
student 1784  1.9  9.8 4617224 392104 ?  Sl   11:40   1:22 /usr/bin/gnome-shell
student 2120  0.4  6.1 3214560 244820 ?  Sl   11:42   0:11 /snap/firefox/...
root     782  0.0  1.1  398736  45212 ?  Ssl  11:39   0:00 /usr/libexec/packagekitd
$ pgrep -a sshd
1012 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
$ sleep 300 &  ## run in background
[1] 2801
$ kill 2801  ## SIGTERM (15)
$ kill -9 PID  ## SIGKILL: last resort
```

نکته

ابزارهای مکمل: `pgrep name` شماره‌ی PID را پیدا می‌کند، `pkill name` و `killall name` بر اساس نام سیگنال می‌فرستند، `jobs` و `fg` و `bg` کارهای پس‌زمینه‌ی همان ترمینال را مدیریت می‌کنند و `Ctrl+Z` برنامه‌ی جاری را متوقف (Suspend) می‌کند.

top

نظارت زنده با top

Live Monitoring

`top` مثل Task Manager در Windows است و هر سه ثانیه فهرست فرایندها را به‌روز می‌کند. سطرهای بالا خلاصه‌ی سیستم‌اند: زمان روشن‌بودن، تعداد کاربران، load average، وضعیت CPU و حافظه. نسخه‌ی رنگی و راحت‌تر آن `htop` است.

کلید در top	کار	کلید در top	کار
`P`	مرتب‌سازی بر اساس CPU	`M`	مرتب‌سازی بر اساس حافظه
`k`	کشتن فرایند (PID می‌پرسد)	`r`	تغییر اولویت (renice)
`1`	نمایش هر هسته جدا	`u`	فیلتر بر اساس کاربر
`h`	راهنما	`q`	خروج

student@ubuntu-lab: ~

```
$ top -b -n1 | head -5  ## batch mode: one snapshot, good for scripts
top - 13:31:02 up  1:51,  1 user,  load average: 0.12, 0.10, 0.08
Tasks: 231 total,   1 running, 230 sleeping,   0 stopped,   0 zombie
%Cpu(s):  2.1 us,  0.8 sy,  0.0 ni, 96.9 id,  0.0 wa,  0.0 hi,  0.2 si,  0.0 st
MiB Mem :   3902.7 total,   1402.3 free,   1201.5 used,   1550.2 buff/cache
MiB Swap:   3904.0 total,   3904.0 free,      0.0 used.   2460.1 avail Mem
$ uptime
 13:31:10 up  1:51,  1 user,  load average: 0.12, 0.10, 0.08
```

نکته‌ی آزمون

load average سه عدد است: میانگین ۱، ۵ و ۱۵ دقیقه. روی یک ماشین ۲ هسته‌ای، عدد ۲ یعنی CPU کاملاً مشغول است. در آزمون بدانید `top` نمای زنده و `ps` عکس لحظه‌ای است، و `kill -9` سیگنال SIGKILL می‌فرستد.

کنترل سرویس با systemctl

Managing Services with systemd

systemctl

ساختار کلی · SYNTAX

```
systemctl [start|stop|restart|reload|status|enable|disable|is-active|is-enabled]
UNIT
```

`systemctl` ابزار اصلی کار با systemd است. دو مفهوم را جدا کنید: **start/stop** وضعیت **الان** سرویس را عوض می‌کند و **enable/disable** مشخص می‌کند سرویس **در بوت بعدی** خودکار اجرا شود یا نه. عبارت `enable --now` هر دو کار را با هم انجام می‌دهد.

زیرفرمان	کار
`status ssh`	وضعیت، PID، حافظه و چند خط آخر لاگ
`start` / `stop`	روشن / خاموش کردن الان
`restart`	خاموش و روشن کردن (قطع اتصال‌ها)
`reload`	خواندن دوباره‌ی تنظیمات بدون قطع سرویس
`enable` / `disable`	اجرای خودکار در بوت: فعال / غیرفعال
`enable --now`	فعال‌سازی و اجرا در یک قدم
`is-active` / `is-enabled`	پاسخ کوتاه برای اسکریپت‌ها
`list-units --type=service`	فهرست سرویس‌های بارگذاری‌شده
`--failed`	سرویس‌هایی که شکست خورده‌اند
`daemon-reload`	بعد از تغییر فایل Unit

student@ubuntu-lab: ~

```
$ systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
     Active: active (running) since Sun 2026-10-04 11:40:02 +0330; 1h 52min ago
TriggeredBy: ● ssh.socket
   Main PID: 1012 (sshd)
$ sudo systemctl restart ssh
$ systemctl is-enabled cron; systemctl is-active cron
enabled
active
$ systemctl --failed
  0 loaded units listed.
```

ssh.socket در Ubuntu 24.04

از Ubuntu 22.10 به بعد، SSH به‌طور پیش‌فرض با **Socket Activation** اجرا می‌شود: `ssh.socket` روی پورت ۲۲ گوش می‌دهد و با اولین اتصال `ssh.service` را روشن می‌کند. برای همین ممکن است `ssh.service` را disabled اما در حال کار ببینید.

journalctl

لاگ‌ها: journalctl، dmesg و /var/log/

Logs: journald, Kernel Ring Buffer and Log Files

سوییچ‌های رایج journalctl

سوییچ	شکل بلند	کاربرد	مثال
-u	--unit	لاگ یک سرویس	journalctl -u ssh
-f	--follow	دنبال‌کردن زنده (مثل tail -f)	journalctl -f
-n N	--lines	N خط آخر	journalctl -n 20
-b	--boot	فقط از بوت فعلی	journalctl -b
-p	--priority	سطح اهمیت: err، warning…	journalctl -p err -b
-k	--dmesg	فقط پیام‌های کرنل	journalctl -k
	--since	از زمان مشخص	--since "1 hour ago"
-r	--reverse	جدیدترین اول	journalctl -r

student@ubuntu-lab: ~

```
$ journalctl -u ssh --since today -n 3 --no-pager
Oct 04 12:10:31 ubuntu-lab sshd[2241]: Failed password for invalid user wronguser from 127.0.0.1
Oct 04 12:10:40 ubuntu-lab sshd[2241]: Connection closed by invalid user wronguser 127.0.0.1
Oct 04 13:33:05 ubuntu-lab systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
$ journalctl -p err -b --no-pager | tail -2
$ sudo dmesg -T | grep -iE 'ens33|e1000' | head -2
[Sun Oct  4 11:39:12 2026] e1000 0000:02:01.0 ens33: renamed from eth0
[Sun Oct  4 11:39:15 2026] e1000: ens33 NIC Link is Up 1000 Mbps Full Duplex
```

فایل لاگ در /var/log	محتوا
syslog	لاگ عمومی سیستم (با rsyslog)
auth.log	ورود، sudo، SSH و احراز هویت
kern.log	پیام‌های کرنل
dpkg.log و apt/	نصب و حذف بسته‌ها
journal/	پایگاه دودویی journald
wtmp / btmp	ورودهای موفق / ناموفق (با last و lastb)

؟ نکته‌ی آزمون

`dmesg` پیام‌های **حلقه‌ی بافر کرنل** را نشان می‌دهد (شناسایی سخت‌افزار و درایورها هنگام بوت). لاگ‌های متنی سنتی در `/var/log` هستند. `journalctl` لاگ‌های systemd را می‌خواند و `-u` برای انتخاب سرویس است.

LAB 20

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

مدیریت سرویس SSH و دیدن لاگ‌ها

چرخه‌ی کامل یک سرویس را تجربه کنید: وضعیت، توقف، اجرا، فعال‌سازی در بوت، بارگذاری مجدد تنظیمات و ردیابی لاگ‌ها و فرایندها.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. وضعیت سرویس SSH و PID فرایند اصلی آن را ببینید.

۲. فرایند sshd را با `ps` پیدا کنید و والد آن را تشخیص دهید.

۳. سرویس را متوقف کنید و نشان دهید پورت ۲۲ دیگر باز نیست؛ دوباره اجرا کنید.

۴. سرویس را در بوت فعال کنید.

۵. در یک ترمینال دیگر لاگ زنده‌ی SSH را دنبال کنید و یک اتصال موفق بزنید.

۶. یک فرایند `sleep` در پس‌زمینه بسازید، با top پیدا کنید و با kill پایانش دهید.

۷. پیام‌های کرنل مربوط به کارت شبکه را پیدا کنید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ systemctl status ssh --no-pager | grep -E 'Active|Main PID'
$ ps -o pid,ppid,user,cmd -C sshd
    PID    PPID USER     CMD
   1012       1 root     sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
$ sudo systemctl stop ssh.socket ssh.service
$ ss -tln | grep ':22 ' || echo 'port 22 closed'
port 22 closed
$ sudo systemctl enable --now ssh
Created symlink /etc/systemd/system/multi-user.target.wants/ssh.service
$ systemctl is-enabled ssh; ss -tln | grep ':22 '
enabled
LISTEN 0  4096   0.0.0.0:22   0.0.0.0:*
```

student@ubuntu-lab: ~

```
$ journalctl -u ssh -f  ## terminal 2 (Ctrl+C to stop)
$ ssh student@localhost 'hostname'  ## terminal 1
ubuntu-lab
## terminal 2 shows: Accepted password for student from 127.0.0.1 ...
$ sleep 600 & echo "PID=$!"
PID=3120
$ top -b -n1 -p 3120 | tail -1
   3120 student   20   0    8296   1920   1792 S   0.0   0.0   0:00.00 sleep
$ kill 3120; jobs
[1]+  Terminated              sleep 600
$ sudo dmesg | grep -i ens3
```

✓ بررسی نتیجه (Verify)

والد فرایند sshd باید (systemd) PID ۱ باشد. بعد از stop، پورت ۲۲ نباید در `ss` دیده شود و بعد از `enable --now` دوباره LISTEN باشد. در لاگ زنده باید خط `Accepted password` ظاهر شود.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. فرق `systemctl restart` و `systemctl reload` چیست؟
2. فرق `enable` و `start` را توضیح دهید.
3. کدام فرمان فقط لاگ‌های یک سرویس را نشان می‌دهد؟
4. فرق `kill` و `kill -9` چیست و چرا -9 آخرین راه است؟
5. پیام‌های شناسایی سخت‌افزار هنگام بوت را با چه فرمانی می‌بینید؟

وزن در آزمون: ۲

Objective 4.5

شبکه و IP

Your Computer on the Network

برای اینکه دو کامپیوتر با هم حرف بزنند، هر کدام به یک **آدرس IP** نیاز دارد. وقتی مقصد بیرون از شبکه‌ی محلی است، بسته‌ها به **Gateway** (معمولاً مودم یا روتر) تحویل داده می‌شوند. نام‌هایی مثل `ubuntu.com` را **DNS** به IP تبدیل می‌کند و آدرس خودکار را **DHCP** می‌دهد.

IPv4	آدرس ۳۲ بیتی در قالب چهار عدد ۰ تا ۲۵۵: `192.168.56.10` .
IPv6	آدرس ۱۲۸ بیتی هگزادسیمال: `fe80::20c:29ff:fe7a:1b2c` .
Netmask / CIDR	مشخص می‌کند چه بخشی از IP مال شبکه است: `/24` یعنی `255.255.255.0` (۲۵۴ میزبان).
Gateway	روتری که بسته‌های مقصد خارج از شبکه را تحویل می‌گیرد؛ Default Route.
DNS	سیستم نام دامنه؛ تبدیل نام به IP. پیکربندی در Ubuntu با systemd-resolved.
DHCP	پروتکلی که IP و Gateway و DNS را خودکار به ماشین می‌دهد.
Port	شماره‌ی سرویس روی یک IP: SSH ۲۲، HTTP ۸۰، HTTPS ۴۴۳، DNS ۵۳.
localhost	آدرس خود ماشین: `127.0.0.1` و `::1` روی کارت `lo` .

≈ تشبیه

IP مثل پلاک خانه است، Netmask مشخص می‌کند کدام بخش پلاک نام محله است و Gateway سر کوچه است که نامه‌های شهرهای دیگر را تحویل می‌گیرد. DNS دفترچه‌ی تلفن است که اسم را به شماره تبدیل می‌کند و DHCP مأمور شهرداری است که به خانه‌ی تازه‌ساز پلاک می‌دهد.

آدرس / بازه	معنی
`10.0.0.0/8` ، `172.16.0.0/12` ، `192.168.0.0/16`	آدرس‌های خصوصی (Private)؛ در اینترنت مسیریابی نمی‌شوند
`127.0.0.0/8`	Loopback؛ خود ماشین
`169.254.0.0/16`	APIPA؛ وقتی DHCP جواب نداده
`fe80::/10`	IPv6 Link-local؛ فقط در همان شبکه‌ی محلی

؟ نکته‌ی آزمون

آدرس‌های خصوصی IPv4، معنی `/24` ، پورت‌های رایج (۲۲، ۸۰، ۴۴۳، ۵۳) و نقش DNS و DHCP و Gateway از پرسش‌های مطمئن آزمون‌اند. همچنین بدانید فرمان‌های قدیمی `ifconfig` ، `route` و `netstat` (از بسته‌ی net-tools) جای خود را به `ip` و `ss` داده‌اند.

آدرس و مسیر: ip addr و ip route

The ip Command

ip

SYNTAX · ساختار کلی

```
ip [-br] [-4|-6] OBJECT COMMAND     # OBJECT: addr | link | route | neigh
```

student@ubuntu-lab: ~

```
$ ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens33            UP             192.168.17.128/24 fe80::20c:29ff:fe7a:1b2c/64
ens37            UP             192.168.56.128/24 fe80::20c:29ff:fe7a:1b36/64
$ ip addr show ens33 | grep inet
    inet 192.168.17.128/24 metric 100 brd 192.168.17.255 scope global dynamic ens33
$ ip route
default via 192.168.17.2 dev ens33 proto dhcp src 192.168.17.128 metric 100
192.168.17.0/24 dev ens33 proto kernel scope link src 192.168.17.128 metric 100
192.168.56.0/24 dev ens37 proto kernel scope link src 192.168.56.128
$ ip -br link  ## link layer: state and MAC
ens33            UP             00:0c:29:7a:1b:2c <BROADCAST,MULTICAST,UP,LOWER_UP>
```

فرمان	کار
`ip a` یا `ip addr`	نمایش آدرس‌ها
`ip -4 a`	فقط IPv4
`ip link set ens37 down`	غیرفعال‌کردن کارت (موقت)
`sudo ip addr add 10.9.9.9/24 dev ens37`	افزودن IP موقت (تا ری‌استارت)
`ip route get 8.8.8.8`	این مقصد از کدام مسیر و کارت می‌رود؟
`ip neigh`	جدول ARP؛ MAC همسایه‌ها

VM

VMware Workstation

در VMware، Gateway شبکه‌ی NAT معمولاً آدرس `.2` همان شبکه است (مثلاً `192.168.17.2`) و روی VMnet8 DHCP و VMnet1 را خود VMware انجام می‌دهد. شبکه‌ی Host-Only هیچ Gateway ای به اینترنت ندارد.

ping

عیب‌یابی: ping، DNS، ss و resolvectl

Testing Connectivity, DNS and Ports

ping -c 4 HOST

بررسی رسیدن به یک IP یا نام (ICMP)

```
$ ping -c 4 192.168.17.2
```

host NAME

پرسش سریع DNS

```
$ host ubuntu.com
```

dig NAME

پرسش کامل DNS با جزئیات

```
$ dig +short ubuntu.com A
```

ss -tulpn

پورت‌های TCP/UDP در حال شنود

```
$ sudo ss -tulpn
```

resolvectl status

DNS هر کارت شبکه

```
$ resolvectl status ens33
```

tracepath HOST

مسیر بسته‌ها تا مقصد

```
$ tracepath -n 1.1.1.1
```

student@ubuntu-lab: ~

```
$ ping -c 2 192.168.17.2
64 bytes from 192.168.17.2: icmp_seq=1 ttl=128 time=0.412 ms
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
$ host ubuntu.com
ubuntu.com has address 185.125.190.20
$ dig +short ubuntu.com
185.125.190.20
$ sudo ss -tulpn | grep LISTEN
tcp LISTEN 0 4096 127.0.0.53%lo:53  0.0.0.0:*  users:(("systemd-resolve",pid=611))
tcp LISTEN 0 4096       0.0.0.0:22  0.0.0.0:*  users:(("sshd",pid=1012))
$ resolvectl status ens33 | grep -E 'DNS Servers|Current DNS'
Current DNS Server: 192.168.17.2
       DNS Servers: 192.168.17.2
$ cat /etc/resolv.conf | grep nameserver
nameserver 127.0.0.53
```

سوییچ ss	معنی
-t / -u	TCP / UDP
-l	فقط سوکت‌های در حال شنود (Listening)
-n	عدد پورت به‌جای نام سرویس
-p	نام و PID برنامه (نیاز به sudo)
-a	همه، شامل اتصال‌های برقرار

نکته

در Ubuntu، فایل `/etc/resolv.conf` به سرویس محلی `127.0.0.53` (systemd-resolved) اشاره می‌کند و DNS واقعی را باید با `resolvectl` ببینید. فایل `/etc/hosts` قبل از DNS بررسی می‌شود و برای نام‌گذاری ماشین‌های آزمایشگاه عالی است.

LAB 21

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

عیب‌یابی شبکه لایه‌به‌لایه

با یک روش منظم از پایین به بالا (کارت، DNS، Gateway، IP، سرویس) مشکل شبکه را پیدا کنید؛ همان کاری که پشتیبان‌های حرفه‌ای انجام می‌دهند.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. لایه‌ی ۱: وضعیت کارت‌ها را با `ip -br link` ببینید.

۲. لایه‌ی ۲: IP و ماسک هر کارت را بررسی کنید.

۳. لایه‌ی ۳: Gateway پیش‌فرض را پیدا و ping کنید.

۴. لایه‌ی ۴: به یک IP اینترنتی ping بزنید تا مسیر بیرون بررسی شود.

۵. لایه‌ی ۵: DNS را با `host` و `dig` و `resolvectl` بیازمایید.

۶. لایه‌ی ۶: بررسی کنید سرویس SSH روی چه پورتی گوش می‌دهد.

۷. خرابی عمدی: کارت NAT را down کنید، ببینید کدام آزمون شکست می‌خورد و برگردانید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ ip -br link | grep -v lo
ens33    UP    00:0c:29:7a:1b:2c <BROADCAST,MULTICAST,UP,LOWER_UP>
ens37    UP    00:0c:29:7a:1b:36 <BROADCAST,MULTICAST,UP,LOWER_UP>
$ ip -4 -br addr
$ GW=$(ip route | awk '/^default/ {print $3}'); echo $GW
192.168.17.2
$ ping -c2 -q $GW && ping -c2 -q 8.8.8.8
$ host -t A archive.ubuntu.com && resolvectl query ubuntu.com
$ sudo ss -tlnp | grep ':22'
```

student@ubuntu-lab: ~

```
$ sudo ip link set ens33 down
$ ping -c1 -W1 8.8.8.8
ping: connect: Network is unreachable
$ host ubuntu.com
;; connection timed out; no servers could be reached
$ sudo ip link set ens33 up && sleep 5 && ping -c1 -q 8.8.8.8
1 packets transmitted, 1 received, 0% packet loss
```

✓ بررسی نتیجه (Verify)

وقتی کارت NAT خاموش است، مسیر پیش‌فرض حذف می‌شود و پیام `Network is unreachable` می‌گیرید؛ یعنی مشکل در لایه‌ی کارت یا مسیر است، نه DNS. بعد از روشن‌کردن دوباره، DHCP در چند ثانیه IP را برمی‌گرداند. اگر برنگشت: `sudo netplan apply` .

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. چرا عیب‌یابی شبکه را از پایین (کارت) به بالا (DNS) انجام می‌دهیم؟
2. اگر `ping 8.8.8.8` کار کند ولی `ping ubuntu.com` نه، مشکل کجاست؟
3. Gateway پیش‌فرض را با کدام فرمان می‌بینید؟
4. سوییچ‌های `-tulpn` در ss هر کدام چه معنایی دارند؟
5. آدرس `127.0.0.53` در `/etc/resolv.conf` چیست؟

وزن در آزمون: ۲

Objective 4.6

تنظیم IP ثابت با netplan

Static IP with netplan and networkd

سرورها معمولاً IP **ثابت** دارند تا همیشه با همان آدرس پیدا شوند. Ubuntu تنظیمات شبکه را در فایل‌های YAML در پوشه‌ی `/etc/netplan/` نگه می‌دارد. **netplan** این فایل‌ها را می‌خواند و برای یکی از دو موتور شبکه (Renderer) ترجمه می‌کند: `networkd` (systemd-networkd، مناسب سرور) یا `NetworkManager` (مناسب دسکتاپ و Wi-Fi).

کلید YAML	معنی	مثال
`network:`	ریشه‌ی تنظیمات	—
`version: 2`	نسخه‌ی قالب netplan (همیشه ۲)	`version: 2`
`renderer:`	موتور شبکه	`networkd` یا `NetworkManager`
`ethernets:`	فهرست کارت‌های سیمی	`ens37:`
`dhcp4:`	دریافت خودکار IPv4	`false` برای IP ثابت
`addresses:`	IP یا IPها با CIDR	`[192.168.56.10/24]`
`routes:`	مسیرها؛ `to: default` یعنی Gateway	`via: 192.168.17.2` / `- to: default`
`nameservers:`	سرورهای DNS و دامنه‌ی جستجو	`addresses: [1.1.1.1, 8.8.8.8]`

> ⚠ **هشدار**
>
> YAML به **تورفتگی** (Indentation) حساس است و Tab را قبول نمی‌کند. فقط از فاصله استفاده کنید (معمولاً ۲ تا برای هر سطح) و بعد از `:` یک فاصله بگذارید. یک فاصله‌ی اشتباه یعنی شبکه بالا نمی‌آید؛ روی سرور راه دور یعنی قطع ارتباط با سرور!

فایل‌های netplan و فرمان‌های آن

student@ubuntu-lab: ~

```
$ ls -l /etc/netplan/
-rw------- 1 root root 104 Apr 22 13:12 01-network-manager-all.yaml
-rw------- 1 root root 211 Oct  4 11:20 50-cloud-init.yaml
$ sudo cat /etc/netplan/01-network-manager-all.yaml
# Let NetworkManager manage all devices on this system
network:
  version: 2
  renderer: NetworkManager
$ sudo netplan get   ## merged configuration of all files
```

فرمان	کار
`sudo netplan generate`	تبدیل YAML به تنظیمات موتور (بدون اعمال)
`sudo netplan try`	اعمال **آزمایشی**؛ اگر در ۱۲۰ ثانیه Enter نزنید، به حالت قبل برمی‌گردد
`sudo netplan apply`	اعمال دائمی تنظیمات
`sudo netplan get` / `status`	نمایش تنظیمات ادغام‌شده / وضعیت کارت‌ها
`networkctl status ens37`	وضعیت کارت از دید systemd-networkd

نکته

فایل‌ها به ترتیب الفبایی خوانده می‌شوند و فایل با عدد بزرگ‌تر بر قبلی غلبه می‌کند. برای تنظیمات خودتان فایل جدیدی مثل `60-hostonly.yaml` بسازید و مجوز آن را `600` بگذارید؛ netplan به فایل‌هایی که همه می‌توانند بخوانند هشدار می‌دهد.

YAML

نمونه‌ی کامل IP ثابت با networkd

A Complete Static Configuration

در آزمایشگاه ما کارت اول (`ens33`، NAT) با DHCP اینترنت می‌گیرد و کارت دوم (`ens37`، Host-Only) باید IP ثابت `192.168.56.10` داشته باشد. چون Host-Only به اینترنت راه ندارد، روی آن Gateway پیش‌فرض نمی‌گذاریم؛ فقط اگر بخواهیم به شبکه‌ی دیگری از طریق یک روتر محلی برسیم، یک مسیر مشخص (`routes`) اضافه می‌کنیم.

/etc/netplan/60-lab.yaml

```
# /etc/netplan/60-lab.yaml  (chmod 600)
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:                  # NAT  → internet via DHCP
      dhcp4: true
    ens37:                  # Host-Only → static lab address
      dhcp4: false
      addresses:
        - 192.168.56.10/24
      routes:               # local gateway for another lab subnet
        - to: 10.10.0.0/24
          via: 192.168.56.1
      nameservers:
        search: [lab.local]
        addresses: [192.168.56.1, 1.1.1.1]
```

اگر سروری فقط یک کارت دارد و IP ثابت با دسترسی اینترنت می‌خواهد، شکل رایج مسیر پیش‌فرض این است:

default route (single-NIC server)

```
      routes:
        - to: default
          via: 192.168.17.2
```

؟ نکته‌ی آزمون

کلیدواژه‌ی قدیمی `gateway4:` منسوخ (Deprecated) شده و netplan جدید به‌جای آن `routes:` با `to: default` می‌خواهد. همچنین بدانید `netplan try` امن‌ترین روش اعمال روی سرور راه دور است، چون در صورت قطع ارتباط خودش برمی‌گردد.

VM VMware Workstation

اگر Renderer را از NetworkManager به networkd عوض کنید، آیکون شبکه‌ی میزکار GNOME کارت را «unmanaged» نشان می‌دهد؛ این طبیعی است. در VMware، از Virtual Network Editor مطمئن شوید بازه‌ی VMnet1 همان `192.168.56.0/24` است و IP ثابتی که انتخاب می‌کنید خارج از بازه‌ی DHCP آن باشد.

LAB 22

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

تنظیم IP ثابت روی کارت Host-Only

کارت Host-Only را با netplan و systemd-networkd روی `192.168.56.10/24` ثابت کنید، آن را با netplan try ایمن اعمال کنید و از Host با SSH وصل شوید.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. از ماشین یک Snapshot با نام `before-lab22` بگیرید.

۲. نام کارت Host-Only و IP فعلی آن را پیدا کنید.

۳. از فایل‌های فعلی netplan نسخه‌ی پشتیبان بگیرید.

۴. فایل `60-lab.yaml` را طبق درس با مجوز ۶۰۰ بسازید.

۵. با `netplan generate` خطای نحوی را بررسی و با `netplan try` اعمال کنید.

۶. IP جدید، مسیرها و DNS را بررسی کنید.

۷. از سیستم Host (Windows) با `ssh student@192.168.56.10` وصل شوید و ری‌استارت کنید تا ماندگاری بررسی شود.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ ip -br -4 addr | grep 192.168.56
ens37   UP   192.168.56.128/24
$ sudo mkdir -p /root/netplan-backup && sudo cp /etc/netplan/*.yaml /root/netplan-backup/
$ sudo nano /etc/netplan/60-lab.yaml  ## type the YAML from the lesson
$ sudo chmod 600 /etc/netplan/60-lab.yaml
$ sudo netplan generate && echo 'syntax OK'
syntax OK
$ sudo netplan try
Do you want to keep these settings?
Press ENTER before the timeout to accept the new configuration
Changes will revert in 118 seconds
Configuration accepted.
```

student@ubuntu-lab: ~

```
$ ip -br -4 addr show ens37
ens37   UP   192.168.56.10/24
$ ip route | grep ens37
10.10.0.0/24 via 192.168.56.1 dev ens37 proto static
192.168.56.0/24 dev ens37 proto kernel scope link src 192.168.56.10
$ networkctl status ens37 | grep -E 'State|Address'
             State: routable (configured)
           Address: 192.168.56.10
$ resolvectl dns ens37
Link 3 (ens37): 192.168.56.1 1.1.1.1
PS C:\> ssh student@192.168.56.10   ## from Windows host (PowerShell)
```

✓ **بررسی نتیجه (Verify)**

بعد از ری‌استارت هم باید `ip -br addr` آدرس `192.168.56.10/24` را روی ens37 نشان دهد و اینترنت از ens33 کار کند (`ping -c1 ubuntu.com`). اگر اشتباه کردید و شبکه قطع شد، فایل را با `sudo rm /etc/netplan/60-lab.yaml` حذف و `sudo netplan apply` کنید یا به Snapshot برگردید.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. فایل‌های تنظیمات netplan در کدام پوشه هستند و چه قالبی دارند؟
2. دو Renderer رایج netplan را نام ببرید و کاربرد هر کدام را بگویید.
3. فرق `netplan try` و `netplan apply` چیست؟
4. چرا روی کارت Host-Only مسیر پیش‌فرض (default) تعریف نکردیم؟
5. جایگزین کلیدواژه‌ی منسوخ `gateway4` چیست؟

EXAM 010-160 · TOPIC 5

5

امنیت و مجوزهای فایل

SECURITY AND FILE PERMISSIONS

لینوکس از روز اول چندکاربره طراحی شد. در این فصل یاد می‌گیرید کاربران و گروه‌ها کجا تعریف می‌شوند، چطور کاربر و گروه بسازید، مجوزهای خواندن و نوشتن و اجرا چطور کار می‌کنند، مجوزهای ویژه و لینک‌ها چه هستند و در پایان با fstab و sudoers یک پوشه‌ی اشتراکی امن می‌سازید.

وزن در آزمون: ۷ سؤال از ۴۰

کاربران و امنیت پایه	5.1
ایجاد کاربران و گروه‌ها	5.2
مجوزها و مالکیت	5.3
مجوزهای ویژه و لینک‌ها	5.4
sudoers و fstab	5.5

وزن در آزمون: ۲

Objective 5.1

کاربران و امنیت پایه

Basic Security and Identifying User Types

هر فرایند و هر فایل در لینوکس متعلق به یک **کاربر** و یک **گروه** است. کرنل کاربر را با نام نمی‌شناسد، بلکه با یک عدد به نام **UID** (User ID)؛ گروه هم **GID** دارد. سه نوع حساب داریم: **root** (مدیر کل با UID صفر)، **کاربران سیستمی** یا سرویس (برای اجرای سرویس‌ها مثل `www-data` و `sshd`) و **کاربران عادی** (انسان‌ها، از ۱۰۰۰ UID به بعد در Ubuntu).

نوع حساب	UID در Ubuntu	شِل	نمونه
root (Superuser)	`0`	`/bin/bash`	`root`
کاربر سیستمی / سرویس	`1` تا `999`	`/usr/sbin/nologin`	`daemon` ، `www-data` ، `syslog`
کاربر عادی	`1000` به بعد	`/bin/bash`	`student`
nobody	`65534`	`nologin`	کاربر بدون امتیاز

student@ubuntu-lab: ~

```
$ whoami
student
$ id
uid=1000(student) gid=1000(student) groups=1000(student),4(adm),24(cdrom),27(sudo),30(dip),
46(plugdev),100(users)
$ id www-data
uid=33(www-data) gid=33(www-data) groups=33(www-data)
$ groups
student adm cdrom sudo dip plugdev users
$ who  ## who is logged in now
student  seat0        2026-10-04 11:40 (login screen)
student  pts/1        2026-10-04 13:40 (192.168.56.1)
$ last -n 3  ## login history
$ w  ## who is logged in and what they are doing
```

تشبیه

root مثل کلید اصلی (Master Key) هتل است که همه‌ی درها را باز می‌کند. کاربران عادی کارت اتاق خودشان را دارند و کاربران سرویس کارکنانی هستند که فقط به آشپزخانه یا رختشویخانه دسترسی دارند. sudo یعنی پذیرش به شما اجازه می‌دهد برای یک کار مشخص، کلید اصلی را چند دقیقه قرض بگیرید.

sudo

تعویض کاربر و اجازه: su و sudo

su and sudo

`su` (Switch User) شِل یک کاربر دیگر را با **رمز همان کاربر** باز می‌کند. `sudo` یک فرمان را با دسترسی root (یا کاربر دیگر) اجرا می‌کند ولی **رمز خود شما** را می‌پرسد و همه‌چیز را در لاگ ثبت می‌کند. در Ubuntu کاربر root رمز ندارد، پس روش استاندارد `sudo` است و فقط اعضای گروه `sudo` اجازه‌ی آن را دارند.

فرمان	کار	رمز چه کسی؟
`su - USER`	ورود کامل به شِل USER با محیط او (`-` یعنی login shell)	رمز USER
`sudo CMD`	اجرای یک فرمان با دسترسی root	رمز خودتان
`sudo -u USER CMD`	اجرای فرمان به‌عنوان کاربر دیگر	رمز خودتان
`sudo -i`	شِل root با محیط root	رمز خودتان
`sudo -l`	چه کارهایی اجازه دارم؟	رمز خودتان
`sudo -k`	فراموش‌کردن رمز ذخیره‌شده (۱۵ دقیقه)	—

student@ubuntu-lab: ~

```
$ sudo -l | tail -2
User student may run the following commands on ubuntu-lab:
    (ALL : ALL) ALL
$ sudo -u www-data whoami
www-data
$ sudo -i
# whoami; pwd
root
/root
# exit
$ sudo grep sudo /var/log/auth.log | tail -1
2026-10-04T13:45:10 ubuntu-lab sudo: student : TTY=pts/0 ; PWD=/home/student ; USER=root ;
COMMAND=/usr/bin/whoami
```

هشدار

کار روزمره را هرگز با root انجام ندهید. یک اشتباه تایپی مثل `rm -rf / home/x` (با یک فاصله‌ی اضافه) با دسترسی root کل سیستم را پاک می‌کند. فقط فرمانی را با sudo اجرا کنید که واقعاً به آن نیاز دارد.

فایل‌های حساب: passwd، shadow و group

Account Database Files

/etc/passwd

اطلاعات کاربران در سه فایل متنی است. `/etc/passwd` را همه می‌توانند بخوانند و رمز در آن نیست (فقط `x`). رمزها به شکل **هش** در `/etc/shadow` هستند که فقط root می‌خواند. گروه‌ها در `/etc/group` (و رمز گروه در `/etc/gshadow`) تعریف می‌شوند.

فیلد /etc/passwd	مثال	معنی
۱. نام کاربری	`student`	نام ورود
۲. رمز	`x`	رمز در shadow است
۳. UID	`1000`	شناسه‌ی کاربر
۴. GID	`1000`	گروه اصلی (Primary)
۵. GECOS	`student,,,`	نام کامل و توضیح
۶. خانه	`/home/student`	پوشه‌ی خانه
۷. شِل	`/bin/bash`	شِل ورود

student@ubuntu-lab: ~

```
$ grep student /etc/passwd
student:x:1000:1000:student,,,:/home/student:/bin/bash
$ ls -l /etc/passwd /etc/shadow /etc/group
-rw-r--r-- 1 root root   1872 Oct  4 11:20 /etc/passwd
-rw-r----- 1 root shadow 1050 Oct  4 11:20 /etc/shadow
-rw-r--r-- 1 root root    880 Oct  4 11:20 /etc/group
$ sudo grep student /etc/shadow | cut -c1-40
student:$y$j9T$Fz2o3H...:20365:0:99999:7:::
$ grep sudo /etc/group
sudo:x:27:student
```

فیلد /etc/shadow	معنی
۱. نام کاربری	همان نام در passwd
۲. هش رمز	`$y$` یعنی yescrypt؛ `!` یا `*` یعنی قفل یا بدون رمز
۳. آخرین تغییر	روز از ۱ ژانویه‌ی ۱۹۷۰
۴ و ۵. حداقل / حداکثر عمر	روزهای مجاز بین تغییر رمز
۶. هشدار	چند روز قبل از انقضا هشدار بده
۷ و ۸. غیرفعال / انقضا	قفل حساب پس از انقضا

؟ نکته‌ی آزمون

پرسش‌های قطعی: UID کاربر root؟ (`0`). رمزهای هش‌شده کجا هستند؟ (`/etc/shadow`). کاربران سیستمی چرا شِل `nologin` دارند؟ (تا کسی نتواند با آنها وارد شود). فرمان‌های `id` ، `who` ، `w` و `last` را هم بشناسید.

LAB
23

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

شناسایی کاربران و ممیزی دسترسی

مثل یک ممیز امنیتی، حساب‌های سیستم را دسته‌بندی کنید، دسترسی sudo را بررسی و تاریخچه‌ی ورودها را مرور کنید.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. هویت فعلی و گروه‌های خودتان را ببینید.

۲. همه‌ی حساب‌های دارای UID صفر را پیدا کنید (باید فقط root باشد).

۳. کاربران عادی و تعداد کاربران سیستمی را جداگانه گزارش کنید.

۴. اعضای گروه `sudo` را فهرست کنید.

۵. مجوز فایل‌های passwd و shadow را مقایسه کنید.

۶. تاریخچه‌ی ورود و آخرین فرمان‌های sudo را در لاگ ببینید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ id -un; id -Gn
student
student adm cdrom sudo dip plugdev users
$ awk -F: '$3==0 {print $1}' /etc/passwd
root
$ awk -F: '$3>=1000 && $3<65534 {print $1}' /etc/passwd
student
$ awk -F: '$3>0 && $3<1000' /etc/passwd | wc -l
31
$ getent group sudo
sudo:x:27:student
$ stat -c '%A %U:%G %n' /etc/passwd /etc/shadow
-rw-r--r-- root:root /etc/passwd
-rw-r----- root:shadow /etc/shadow
$ last -n 5; sudo grep 'COMMAND' /var/log/auth.log | tail -3
```

✓ بررسی نتیجه (Verify)

فقط یک حساب با UID صفر (root) باید وجود داشته باشد؛ هر حساب دیگر با UID صفر یک **در پشتی** امنیتی است. فایل shadow نباید برای «others» قابل خواندن باشد.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. سه نوع حساب کاربری در لینوکس را با بازه‌ی UID آنها نام ببرید.
2. فرق `su -` و `sudo -i` چیست؟
3. چرا رمز در `/etc/passwd` نیست؟
4. علامت `!` در فیلد دوم shadow چه معنایی دارد؟
5. کدام فرمان کاربران وارد شده و فعالیت آنها را نشان می‌دهد؟

Objective 5.2

وزن در آزمون: ۲

ایجاد کاربران و گروه‌ها

Creating Users and Groups

Ubuntu دو ابزار برای ساخت کاربر دارد: `adduser` اسکریپتی **تعاملی** و دوستانه است که پوشه‌ی خانه می‌سازد، رمز و اطلاعات را می‌پرسد و همه‌چیز را تنظیم می‌کند. `useradd` ابزار **سطح‌پایین** و استاندارد همه‌ی توزیع‌هاست که فقط همان کاری را می‌کند که با سوییچ‌ها بگویید؛ برای اسکریپت‌ها مناسب‌تر است.

student@ubuntu-lab: ~

```
$ sudo adduser ali
info: Adding user `ali' ...
info: Adding new group `ali' (1001) ...
info: Adding new user `ali' (1001) with group `ali (1001)' ...
info: Creating home directory `/home/ali' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Full Name []: Ali Ahmadi
Is the information correct? [Y/n] y
```

سوییچ‌های رایج `useradd`

سوییچ	شکل بلند	کاربرد	مثال
`-m`	`--create-home`	ساخت پوشه‌ی خانه (پیش‌فرض نمی‌سازد!)	`useradd -m sara`
`-s`	`--shell`	تعیین شِل	`-s /bin/bash`
`-c`	`--comment`	نام کامل (GECOS)	`-c "Sara Rad"`
`-G`	`--groups`	گروه‌های تکمیلی	`-G sudo,devs`
`-g`	`--gid`	گروه اصلی	`-g devs`
`-u`	`--uid`	تعیین UID	`-u 1500`
`-d`	`--home-dir`	مسیر خانه‌ی دلخواه	`-d /srv/sara`
`-e`	`--expiredate`	تاریخ انقضای حساب	`-e 2026-12-31`
`-r`	`--system`	ساخت کاربر سیستمی	`useradd -r appsvc`

student@ubuntu-lab: ~

```
$ sudo useradd -m -s /bin/bash -c "Sara Rad" sara
$ sudo passwd sara
New password:
passwd: password updated successfully
$ grep sara /etc/passwd; ls -ld /home/sara
sara:x:1002:1002:Sara Rad:/home/sara:/bin/bash
drwxr-x--- 2 sara sara 4096 Oct  4 13:52 /home/sara
```

نکته

محتوای پوشه‌ی `/etc/skel` (مثل `.bashrc` و `.profile`) هنگام ساخت خانه در پوشه‌ی کاربر جدید کپی می‌شود. اگر می‌خواهید همه‌ی کاربران جدید یک فایل خوش‌آمد داشته باشند، آن را در `/etc/skel` بگذارید. مقادیر پیش‌فرض useradd در `/etc/default/useradd` و `/etc/login.defs` هستند.

usermod

گروه‌ها، تغییر و حذف کاربر

groupadd, usermod, passwd, userdel

فرمان	کار
`sudo groupadd devs`	ساخت گروه جدید
`sudo groupadd -g 2000 ops`	گروه با GID مشخص
`sudo usermod -aG devs ali`	افزودن ali به گروه devs (**حتماً با `-a`**)
`sudo gpasswd -d ali devs`	حذف ali از گروه devs
`sudo usermod -s /bin/zsh ali`	تغییر شِل
`sudo usermod -L ali` / `-U`	قفل / بازکردن حساب
`sudo passwd ali`	تغییر رمز کاربر دیگر
`passwd`	تغییر رمز خودتان
`sudo passwd -e ali`	اجبار به تغییر رمز در ورود بعدی
`sudo chage -l ali`	اطلاعات عمر رمز
`sudo userdel -r ali`	حذف کاربر **همراه با** پوشه‌ی خانه
`sudo groupdel devs`	حذف گروه

⚠ **هشدار**

`usermod -G devs ali` (بدون `-a`) همه‌ی گروه‌های تکمیلی قبلی ali را **حذف** می‌کند و فقط devs را باقی می‌گذارد. اگر این کار را روی خودتان انجام دهید، از گروه sudo بیرون می‌افتید! همیشه `-aG` (append) بنویسید.

student@ubuntu-lab: ~

```
$ sudo groupadd devs
$ sudo usermod -aG devs ali; sudo usermod -aG devs sara
$ getent group devs
devs:x:1003:ali,sara
$ id ali
uid=1001(ali) gid=1001(ali) groups=1001(ali),1003(devs)
$ sudo usermod -L sara && sudo passwd -S sara
sara L 2026-10-04 0 99999 7 -1
```

؟ **نکته‌ی آزمون**

عضویت جدید در گروه **بعد از ورود دوباره** اعمال می‌شود (یا با `newgrp devs`). در آزمون بدانید: گروه اصلی با `-g` و تکمیلی با `-G`، و `userdel` به‌تنهایی پوشه‌ی خانه را پاک نمی‌کند؛ برای آن `-r` لازم است.

LAB 24

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

ایجاد کاربر پروژه‌ای و گروه

برای تیم توسعه‌ی یک پروژه، کاربر و گروه بسازید، عضویت‌ها را تنظیم کنید و یک حساب موقت با تاریخ انقضا تعریف کنید.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. گروه `webteam` بسازید.

۲. کاربر `ali` را با `adduser` و کاربر `sara` را با `useradd` (با خانه، bash و نام کامل) بسازید.

۳. هر دو را به گروه `webteam` اضافه کنید، بدون اینکه گروه‌های قبلی‌شان حذف شود.

۴. برای sara رمز بگذارید و او را مجبور کنید در ورود اول رمز را عوض کند.

۵. کاربر موقت `intern` با انقضای پایان سال بسازید.

۶. با `su - ali` وارد شوید و عضویت را بررسی کنید.

۷. کاربر intern را همراه با خانه‌اش حذف کنید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ sudo groupadd webteam
$ sudo adduser ali  ## answer the questions
$ sudo useradd -m -s /bin/bash -c "Sara Rad" sara
$ sudo usermod -aG webteam ali && sudo usermod -aG webteam sara
$ sudo passwd sara && sudo passwd -e sara
$ sudo useradd -m -e 2026-12-31 -c "Summer intern" intern
$ sudo chage -l intern | grep "Account expires"
Account expires                                         : Dec 31, 2026
$ getent group webteam
webteam:x:1004:ali,sara
$ su - ali
$ id -Gn
ali users webteam
$ exit
$ sudo userdel -r intern && ls /home
ali  sara  student
```

✓ بررسی نتیجه (Verify)

`getent group webteam` باید هر دو کاربر را نشان دهد. در اولین ورود sara پیام `You are required to change your password immediately` می‌آید. بعد از `userdel -r` پوشه‌ی `/home/intern` دیگر وجود ندارد.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. فرق `adduser` و `useradd` در Ubuntu چیست؟
2. چرا در `usermod` باید `-a` را همراه `-G` بنویسیم؟
3. پوشه‌ی `/etc/skel` چه نقشی دارد؟
4. چطور یک حساب را بدون حذف، موقتاً قفل کنیم؟
5. گروه اصلی و گروه تکمیلی چه فرقی دارند؟

Objective 5.3

وزن در آزمون: ۲

مجوزها و مالکیت فایل‌ها

Managing File Permissions and Ownership

هر فایل یک **مالک** (User)، یک **گروه** (Group) و سه دسته مجوز دارد: برای مالک (u)، برای اعضای گروه (g) و برای بقیه (o، others). هر دسته سه مجوز دارد: خواندن (r)، نوشتن (w) و اجرا (x). این مجوزها را در ستون اول `ls -l` می‌بینید.

student@ubuntu-lab: ~

```
$ ls -l report.txt ~/bin/backup.sh
-rw-r--r-- 1 student student  812 Oct  4 14:01 report.txt
-rwxr-xr-x 1 student student 1290 Oct  4 13:18 /home/student/bin/backup.sh
$ ls -ld /etc /tmp
drwxr-xr-x 140 root root 12288 Oct  4 11:20 /etc
drwxrwxrwt  18 root root  4096 Oct  4 14:00 /tmp
```

بخش	d	rwx	r-x	r-x
معنی	نوع فایل	مالک (u)	گروه (g)	بقیه (o)
در مثال	دایرکتوری	خواندن، نوشتن، اجرا	خواندن، اجرا	خواندن، اجرا

مجوز	روی فایل	روی دایرکتوری	عدد
`r`	خواندن محتوا	فهرست‌کردن نام‌ها (`ls`)	`4`
`w`	تغییر محتوا	ساخت، حذف و تغییر نام فایل‌های داخل	`2`
`x`	اجرا به‌عنوان برنامه	ورود به آن (`cd`) و دسترسی به فایل‌ها	`1`

نوع (نویسه‌ی اول)	معنی
`-`	فایل عادی
`d`	دایرکتوری
`l`	لینک نمادی
`c` / `b`	دستگاه کاراکتری / بلوکی (`/dev`)
`s` / `p`	سوکت / Pipe نام‌دار

نکته‌ی آزمون

مجوز `w` روی **دایرکتوری** یعنی می‌توانید فایل‌های داخلش را حذف کنید، حتی اگر روی خود فایل مجوز نوشتن نداشته باشید! همچنین برای ورود به دایرکتوری `x` لازم است و `r` به‌تنهایی فقط فهرست نام‌ها را می‌دهد.

chmod

تغییر مجوز: روش عددی و نمادی

chmod: Octal and Symbolic Modes

SYNTAX · ساختار کلی

```
chmod [-R] MODE FILE...      # MODE: 755 | u+x | g-w | o= | a+r
```

در **روش عددی** برای هر دسته جمع مقادیر را می‌نویسید: r=۴، w=۲، x=۱. پس rwx=۷، rw-=۶، r-x=۵، r--=۴ و ---=۰. سه رقم به ترتیب برای مالک، گروه و بقیه است.

عدد	نمادی	کاربرد رایج
755	rwxr-xr-x	اسکریپت‌ها، برنامه‌ها، دایرکتوری‌های عمومی
644	rw-r--r--	فایل‌های عادی، صفحات وب، تنظیمات عمومی
600	rw-------	فایل‌های خصوصی: کلید SSH، فایل netplan
700	rwx------	دایرکتوری خصوصی مثل ~/.ssh
750	rwxr-x---	دایرکتوری پروژه‌ی تیمی (بدون دسترسی دیگران)
664 / 775	rw-rw-r-- / rwxrwxr-x	همکاری گروهی
777	rwxrwxrwx	**تقریباً هرگز!** همه می‌توانند همه‌کار کنند

در **روش نمادی** می‌گویید برای چه کسی (u g o a) چه عملی (+ افزودن، - حذف، = تنظیم دقیق) روی کدام مجوز (r w x) انجام شود. این روش برای تغییر یک مجوز بدون دست‌زدن به بقیه عالی است.

student@ubuntu-lab: ~

```
$ chmod 600 secret.txt; ls -l secret.txt
-rw------- 1 student student 20 Oct  4 14:05 secret.txt
$ chmod u+x,g-w,o= script.sh; ls -l script.sh
-rwxr----- 1 student student 64 Oct  4 14:05 script.sh
$ chmod a+r script.sh  ## everyone can read
$ chmod -R g+rwX project/  ## X: execute only for dirs (and already-executable files)
$ stat -c "%a %A %n" secret.txt script.sh
600 -rw------- secret.txt
744 -rwxr--r-- script.sh
```

هشدار

chmod -R 777 راه‌حل مشکل دسترسی نیست؛ یک حفره‌ی امنیتی است. مشکل را با مالکیت درست (chown) و گروه مشترک حل کنید. حرف بزرگ X در chmod -R هم کمک می‌کند فقط دایرکتوری‌ها مجوز ورود بگیرند، نه همه‌ی فایل‌ها.

chown

مالکیت و مجوز پیش‌فرض: chown، chgrp و umask

Ownership and Default Permissions

فرمان	کار
`sudo chown ali file`	تغییر مالک
`sudo chown ali:webteam file`	تغییر مالک و گروه با هم
`sudo chown :webteam file`	فقط تغییر گروه
`sudo chown -R ali:webteam dir/`	بازگشتی برای کل پوشه
`chgrp webteam file`	تغییر گروه (مالک می‌تواند به گروه‌هایی که خودش عضو است تغییر دهد)

`umask` مشخص می‌کند فایل‌ها و دایرکتوری‌های **جدید** با چه مجوزی ساخته شوند. مجوز پایه برای فایل `666` و برای دایرکتوری `777` است و umask بیت‌هایش را از آن **کم** می‌کند. umask پیش‌فرض Ubuntu `0002` برای کاربران عادی است، پس فایل‌ها با `664` و دایرکتوری‌ها با `775` ساخته می‌شوند.

umask	فایل جدید	دایرکتوری جدید	کاربرد
`0002`	`664 rw-rw-r--`	`775 rwxrwxr-x`	پیش‌فرض کاربران Ubuntu
`0022`	`644 rw-r--r--`	`755 rwxr-xr-x`	پیش‌فرض root و بیشتر توزیع‌ها
`0027`	`640 rw-r-----`	`750 rwxr-x---`	سرورهای امن‌تر
`0077`	`600 rw-------`	`700 rwx------`	کاملاً خصوصی

student@ubuntu-lab: ~

```
$ umask; umask -S
0002
u=rwx,g=rwx,o=rx
$ umask 027; touch a.txt; mkdir d; ls -ld a.txt d
-rw-r----- 1 student student    0 Oct  4 14:10 a.txt
drwxr-x--- 2 student student 4096 Oct  4 14:10 d
$ sudo chown ali:webteam a.txt; ls -l a.txt
-rw-r----- 1 ali webteam 0 Oct  4 14:10 a.txt
```

نکته‌ی آزمون

umask هیچ‌وقت به فایل جدید مجوز اجرا نمی‌دهد، چون پایه‌ی فایل `666` است. محاسبه‌ی رایج آزمون: umask `022` ← فایل `644` و دایرکتوری `755` . تغییر umask با فرمان فقط در همان شِل است؛ برای ماندگاری آن را در `~/.bashrc` بنویسید.

LAB 25

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

تنظیم مجوزهای صحیح فایل‌ها و دایرکتوری‌ها

برای یک سایت نمونه و یک پوشه‌ی خصوصی، مالکیت و مجوزها را طوری تنظیم کنید که هر کس فقط به همان چیزی که باید، دسترسی داشته باشد.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. پوشه‌ی `/srv/site` را با زیرپوشه‌های `html` و `config` و چند فایل بسازید.

۲. مالکیت کل پوشه را `ali:webteam` کنید.

۳. دایرکتوری‌ها `2775` ... فعلاً `775`، فایل‌های html `664` و فایل config `640` شوند.

۴. یک پوشه‌ی کاملاً خصوصی `~/private` با مجوز ۷۰۰ و فایل ۶۰۰ بسازید.

۵. با کاربر sara آزمایش کنید چه چیزی را می‌تواند بخواند و چه چیزی را نه.

۶. نشان دهید با umask `077` فایل‌های جدید خصوصی ساخته می‌شوند.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ sudo mkdir -p /srv/site/{html,config}
$ echo "<h1>Hi</h1>" | sudo tee /srv/site/html/index.html >/dev/null
$ echo "db_pass=S3cret" | sudo tee /srv/site/config/app.conf >/dev/null
$ sudo chown -R ali:webteam /srv/site
$ sudo find /srv/site -type d -exec chmod 775 {} +
$ sudo chmod 664 /srv/site/html/index.html; sudo chmod 640 /srv/site/config/app.conf
$ sudo chmod o-rx /srv/site/config
$ ls -lR /srv/site | grep -v total
/srv/site:
drwxrwxr-x 2 ali webteam 4096 Oct  4 14:20 html
drwxrwx--- 2 ali webteam 4096 Oct  4 14:20 config
```

student@ubuntu-lab: ~

```
$ mkdir -m 700 ~/private && (umask 077; echo key > ~/private/key.txt)
$ stat -c '%a %n' ~/private ~/private/key.txt
700 /home/student/private
600 /home/student/private/key.txt
$ sudo -u sara cat /srv/site/html/index.html   ## sara is in webteam
<h1>Hi</h1>
$ sudo -u sara cat /srv/site/config/app.conf
db_pass=S3cret
$ sudo -u nobody cat /srv/site/config/app.conf
cat: /srv/site/config/app.conf: Permission denied
```

✓ بررسی نتیجه (Verify)

کاربر sara (عضو webteam) باید هر دو فایل را بخواند، اما کاربر `nobody` (بیرون از گروه) فایل config را نبیند. `~/private` باید `700` و فایل داخلش `600` باشد. اگر sara هنوز دسترسی ندارد، یک بار از حساب خارج و دوباره وارد شود تا عضویت گروه اعمال شود.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. مجوز `640` را به شکل نمادی بنویسید و توضیح دهید.
2. با umask `027` فایل و دایرکتوری جدید چه مجوزی دارند؟
3. فرمان `chmod u+x,g-w,o=` دقیقاً چه کاری می‌کند؟
4. چرا `find -type d -exec chmod 775` بهتر از `chmod -R 775` است؟
5. فرق `chown ali:webteam` و `chgrp webteam` چیست؟

وزن در آزمون: ۱

Objective 5.4

مجوزهای ویژه و لینک‌ها

Special Directories, Files and Links

علاوه بر rwx، سه بیت ویژه وجود دارد که در رقم **چهارم** (اول از چپ) مجوز عددی نوشته می‌شوند. این بیت‌ها رفتار اجرای برنامه‌ها و ساخت فایل در پوشه‌های اشتراکی را تغییر می‌دهند.

بیت	عدد	روی فایل اجرایی	روی دایرکتوری	در ls
setuid (SUID)	`4000`	اجرا با دسترسی **مالک فایل** (نه اجراکننده)	—	`rws` در بخش مالک
setgid (SGID)	`2000`	اجرا با دسترسی گروه فایل	فایل‌های جدید **گروه دایرکتوری** را می‌گیرند	`rws` در بخش گروه
sticky bit	`1000`	—	فقط **مالک** فایل می‌تواند آن را حذف کند	`t` در بخش others

student@ubuntu-lab: ~

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 64152 May 30 12:00 /usr/bin/passwd
$ ls -ld /tmp
drwxrwxrwt 18 root root 4096 Oct  4 14:00 /tmp
$ stat -c '%a %A %n' /usr/bin/passwd /tmp
4755 -rwsr-xr-x /usr/bin/passwd
1777 drwxrwxrwt /tmp
$ sudo find /usr/bin -perm -4000 | head -4  ## all SUID programs
/usr/bin/passwd
/usr/bin/sudo
/usr/bin/su
/usr/bin/mount
```

≈ تشبیه

SUID روی `passwd` مثل کارت ویژه‌ای است که به مهمان هتل اجازه می‌دهد فقط برای عوض‌کردن رمز کارت خودش وارد دفتر مدیریت شود. کاربر عادی نمی‌تواند `/etc/shadow` را بنویسد، اما برنامه‌ی passwd با دسترسی root این کار را برایش انجام می‌دهد. Sticky bit روی `/tmp` هم مثل کمدهای باشگاه است: همه می‌توانند کمد بگیرند ولی هیچ‌کس قفل کمد دیگری را نمی‌شکند.

student@ubuntu-lab: ~

```
$ sudo chmod 2775 /srv/site       ## setgid: new files inherit group webteam
$ sudo chmod g+s /srv/site          ## same, symbolic
$ sudo chmod +t /srv/share          ## sticky bit
$ sudo chmod 1777 /srv/share        ## same, octal
$ chmod u+s file                    ## setuid (dangerous!)
```

نکته

حرف کوچک `s` یا `t` یعنی بیت ویژه **و** مجوز x هر دو فعال‌اند؛ حرف بزرگ `S` یا `T` یعنی بیت ویژه فعال است ولی x نیست (معمولاً یک اشتباه پیکربندی).

هشدار

SUID روی اسکریپت‌ها در لینوکس نادیده گرفته می‌شود و روی برنامه‌های دلخواه یک خطر جدی امنیتی است. مهاجمان بعد از نفوذ اولین کاری که می‌کنند جستجوی فایل‌های SUID است. فهرست آنها را با `find / -perm -4000 2>/dev/null` دوره‌ای بررسی کنید.

ln

stat و لینک‌ها: نمادی و سخت

stat, Symbolic and Hard Links

هر فایل در لینوکس یک **inode** دارد: ساختاری که مجوزها، مالک، اندازه، زمان‌ها و محل داده روی دیسک را نگه می‌دارد. نام فایل فقط برچسبی است که در دایرکتوری به inode اشاره می‌کند. فرمان `stat` همه‌ی اطلاعات inode را نشان می‌دهد.

student@ubuntu-lab: ~

```
$ stat report.txt
  File: report.txt
  Size: 812        Blocks: 8          IO Block: 4096   regular file
Device: 8,2      Inode: 1048732     Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/ student)   Gid: ( 1000/ student)
Modify: 2026-10-04 14:01:12.000000000 +0330
$ stat -c '%i %h %n' report.txt   ## inode and link count
```

ویژگی	لینک نمادی (`ln -s`)	لینک سخت (`ln`)
ماهیت	فایل کوچکی که **مسیر** مقصد را نگه می‌دارد	یک **نام دیگر** برای همان inode
inode	inode جدا	همان inode اصلی
اگر اصل حذف شود	لینک **شکسته** می‌شود (Dangling)	داده می‌ماند تا آخرین لینک حذف شود
بین فایل‌سیستم‌ها	بله	خیر؛ فقط در همان فایل‌سیستم
به دایرکتوری	بله	خیر
در `ls -l`	`l` و `name → target`	فایل عادی؛ شمارنده‌ی لینک افزایش می‌یابد

student@ubuntu-lab: ~

```
$ echo 'v1' > original.txt
$ ln -s original.txt soft.txt; ln original.txt hard.txt
$ ls -li original.txt soft.txt hard.txt
1048801 -rw-rw-r-- 2 student student  3 Oct  4 14:30 hard.txt
1048801 -rw-rw-r-- 2 student student  3 Oct  4 14:30 original.txt
1048805 lrwxrwxrwx 1 student student 12 Oct  4 14:30 soft.txt → original.txt
$ rm original.txt
$ cat hard.txt; cat soft.txt
v1
cat: soft.txt: No such file or directory
$ readlink -f /bin/sh
/usr/bin/dash
```

نکته‌ی آزمون

در آزمون حتماً می‌پرسند: «کدام لینک بعد از حذف فایل اصلی هم کار می‌کند؟» (Hard). «کدام بین پارتیشن‌ها کار می‌کند؟» (Symbolic). ترتیب آرگومان را هم به خاطر بسپارید: `ln -s TARGET LINK_NAME` ؛ اول مقصد، بعد نام لینک (مثل `cp`).

LAB 26

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

تنظیم مجوزهای ویژه و لینک‌ها

یک پوشه‌ی تیمی با setgid و sticky bit بسازید، فایل‌های SUID سیستم را ممیزی کنید و رفتار لینک‌های نمادی و سخت را آزمایش کنید.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. پوشه‌ی `/srv/team` را با گروه `webteam`، مجوز `2770` بسازید.

۲. به‌عنوان ali فایلی بسازید و نشان دهید گروه آن خودکار `webteam` است.

۳. sticky bit را هم اضافه کنید (مجوز `3770`) و نشان دهید sara نمی‌تواند فایل ali را حذف کند.

۴. همه‌ی فایل‌های SUID در `/usr/bin` را فهرست کنید.

۵. یک لینک نمادی و یک لینک سخت از یک فایل بسازید و inode آنها را مقایسه کنید.

۶. فایل اصلی را حذف کنید و رفتار هر دو لینک را ببینید.

۷. یک میان‌بر نمادی `~/site` به `/srv/site/html` بسازید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ sudo mkdir /srv/team && sudo chown root:webteam /srv/team && sudo chmod 2770 /srv/team
$ sudo -u ali touch /srv/team/ali-notes.txt
$ ls -l /srv/team
-rw-rw-r-- 1 ali webteam 0 Oct  4 14:40 ali-notes.txt
$ sudo chmod 3770 /srv/team; ls -ld /srv/team
drwxrws--T 2 root webteam 4096 Oct  4 14:40 /srv/team
$ sudo -u sara rm /srv/team/ali-notes.txt
rm: cannot remove '/srv/team/ali-notes.txt': Operation not permitted
$ find /usr/bin -perm -4000 -printf '%m %p\n'
4755 /usr/bin/passwd
4755 /usr/bin/sudo
```

student@ubuntu-lab: ~

```
$ cd ~ && echo data > f.txt && ln -s f.txt f.soft && ln f.txt f.hard
$ stat -c '%i %h %F %n' f.txt f.soft f.hard
1048900 2 regular file f.txt
1048903 1 symbolic link f.soft
1048900 2 regular file f.hard
$ rm f.txt; cat f.hard; ls -l f.soft
data
lrwxrwxrwx 1 student student 5 Oct  4 14:45 f.soft → f.txt
$ ln -s /srv/site/html ~/site && ls ~/site
index.html
```

بررسی نتیجه (Verify)

گروه فایل ساخته‌شده توسط ali باید `webteam` باشد، نه `ali` . حرف `T` بزرگ در `drwxrws--T` یعنی sticky فعال است ولی others مجوز x ندارد که اینجا عمدی است. لینک سخت بعد از حذف اصل هنوز `data` را نشان می‌دهد ولی لینک نمادی شکسته است (معمولاً با رنگ قرمز در ls).

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. setgid روی یک دایرکتوری چه اثری دارد؟
2. چرا `/tmp` مجوز `1777` دارد؟
3. فرق `S` بزرگ و `s` کوچک در خروجی `ls -l` چیست؟
4. چرا لینک سخت به دایرکتوری یا بین دو پارتیشن ممکن نیست؟
5. کدام فرمان inode یک فایل را نشان می‌دهد؟ دو روش بنویسید.

Objective 5.5

وزن در آزمون: ۱

fstab و sudoers

Persistent Mounts and sudo Policy

دو فایل تنظیمات وجود دارد که هر مدیر سیستمی باید با احتیاط کامل با آنها کار کند. `/etc/fstab` مشخص می‌کند در هر بوت کدام فایل‌سیستم‌ها کجا و با چه گزینه‌هایی mount شوند. `/etc/sudoers` مشخص می‌کند چه کسی با sudo چه کاری می‌تواند بکند. اشتباه در اولی می‌تواند جلوی بوت سیستم را بگیرد و اشتباه در دومی می‌تواند دسترسی مدیریتی را قفل کند.

فایل /etc/fstab

/etc/fstab

```
# <file system>                    <mount point> <type> <options>              <dump>
<pass>
UUID=9b1c3d2e-...-e7a2             /             ext4   defaults               0
1
/swap.img                          none          swap   sw                     0
0
UUID=4F2A-91C0                     /data         ext4   defaults,nofail        0
2
/srv/team-data.img                 /srv/shared   ext4   loop,nodev,nosuid,noexec 0
2
```

ستون	معنی
۱. دستگاه	`UUID=...` (توصیه‌شده، با `blkid` یا `lsblk -f`)، یا `/dev/sdb1`، یا `LABEL=`
۲. نقطه‌ی mount	پوشه‌ی مقصد؛ باید از قبل وجود داشته باشد
۳. نوع	`ext4`، `xfs`، `vfat`، `swap`، `nfs`…
۴. گزینه‌ها	`defaults`، `ro`، `noexec`، `nosuid`، `nodev`، `nofail`، `user`
۵. dump	معمولاً `0`
۶. pass	ترتیب بررسی fsck: `1` برای ریشه، `2` بقیه، `0` بدون بررسی

student@ubuntu-lab: ~

```
$ sudo blkid /dev/sda2
/dev/sda2: UUID="9b1c3d2e-...-e7a2" BLOCK_SIZE="4096" TYPE="ext4"
$ findmnt /  ## what is mounted on /
TARGET SOURCE    FSTYPE OPTIONS
/      /dev/sda2 ext4   rw,relatime
$ sudo mount -a  ## mount everything in fstab (test after editing!)
$ sudo findmnt --verify  ## check fstab syntax
```

هشدار

بعد از هر تغییر در fstab، **قبل از ری‌استارت** `sudo findmnt --verify` و `sudo mount -a` را اجرا کنید. اگر خطا داد و ری‌استارت کنید، سیستم ممکن است در حالت Emergency بالا بیاید. برای دیسک‌های اختیاری (مثل دیسک USB) گزینه‌ی `nofail` بگذارید.

visudo

sudoers و visudo: ویرایش امن مجوز sudo

Editing sudo Rules Safely

فایل `/etc/sudoers` را **هرگز** مستقیم با nano یا vim باز نکنید. فرمان `visudo` فایل را قفل می‌کند، در ویرایشگر باز می‌کند و قبل از ذخیره **نحو آن را بررسی** می‌کند؛ اگر خطا باشد اجازه‌ی ذخیره نمی‌دهد. روش تمیزتر، ساخت فایل جداگانه در `/etc/sudoers.d/` با `visudo -f` است.

SYNTAX · ساختار کلی

```
WHO   WHERE=(AS_USER:AS_GROUP)   [NOPASSWD:] COMMANDS
%group  ALL=(ALL:ALL) ALL
```

sudoers

```
# /etc/sudoers (default lines in Ubuntu)
root    ALL=(ALL:ALL) ALL
%admin  ALL=(ALL) ALL
%sudo   ALL=(ALL:ALL) ALL
@includedir /etc/sudoers.d

# /etc/sudoers.d/webteam  (created with: sudo visudo -f /etc/sudoers.d/webteam)
%webteam ALL=(root) /usr/bin/systemctl restart nginx, /usr/bin/systemctl status nginx
sara     ALL=(root) NOPASSWD: /usr/bin/journalctl
```

بخش	معنی در `%webteam ALL=(root) /usr/bin/systemctl restart nginx`
`%webteam`	گروه webteam (علامت `%` یعنی گروه؛ بدون آن یعنی کاربر)
`ALL=`	روی همه‌ی میزبان‌ها (برای sudoers مشترک بین چند سرور)
`(root)`	فرمان به‌عنوان root اجرا می‌شود
`/usr/bin/systemctl restart nginx`	**فقط همین** فرمان، با مسیر کامل
`NOPASSWD:`	بدون پرسیدن رمز (با احتیاط)

؟ نکته‌ی آزمون

در آزمون بدانید: `visudo` ابزار درست ویرایش sudoers است، گروه مدیران در Ubuntu `sudo` و در خانواده‌ی Red Hat `wheel` است، و `sudo -l` مجوزهای کاربر را نشان می‌دهد.

! نکته

همیشه یک ترمینال root (`sudo -i`) باز نگه دارید وقتی sudoers را تغییر می‌دهید. اگر اشتباه کردید و sudo از کار افتاد، از همان ترمینال اصلاح می‌کنید. در VMware هم Snapshot قبل از کار نجات‌دهنده است.

LAB 27

لابراتوار عملی · Ubuntu 24.04 LTS روی VMware Workstation

تنظیم پوشه‌ی اشتراکی امن

همه‌ی مهارت‌های فصل را ترکیب کنید: یک فایل‌سیستم جداگانه بسازید، آن را با fstab و گزینه‌های امن دائمی mount کنید، با setgid و sticky بین اعضای تیم به اشتراک بگذارید و به تیم اجازه‌ی محدود sudo بدهید.

■ سناریو: کارهایی که باید خودتان انجام دهید

۱. یک Snapshot با نام `before-lab27` بگیرید.

۲. یک فایل‌سیستم ۲۰۰ ext4 مگابایتی درون فایل `/srv/team-data.img` بسازید.

۳. آن را در `/srv/shared` با گزینه‌های `nodev,nosuid,noexec` در fstab تعریف کنید و آزمایش کنید.

۴. مالکیت را `root:webteam` و مجوز را `3770` کنید.

۵. نشان دهید اسکریپت‌ها در این پوشه اجرا نمی‌شوند (noexec).

۶. با `visudo -f` به گروه webteam فقط اجازه‌ی دیدن لاگ‌های SSH را بدهید.

۷. ری‌استارت کنید و ماندگاری mount را بررسی کنید.

■ راه‌حل گام‌به‌گام

student@ubuntu-lab: ~

```
$ sudo fallocate -l 200M /srv/team-data.img
$ sudo mkfs.ext4 -q /srv/team-data.img
$ sudo mkdir -p /srv/shared
$ echo '/srv/team-data.img /srv/shared ext4 loop,nodev,nosuid,noexec 0 2' | sudo tee -a /etc/
fstab
$ sudo findmnt --verify && sudo mount -a
$ findmnt /srv/shared
TARGET      SOURCE     FSTYPE OPTIONS
/srv/shared /dev/loop3 ext4   rw,nosuid,nodev,noexec,relatime
$ sudo chown root:webteam /srv/shared && sudo chmod 3770 /srv/shared
```

student@ubuntu-lab: ~

```
$ printf '#!/bin/bash\necho hi\n' | sudo -u ali tee /srv/shared/t.sh >/dev/null
$ sudo -u ali chmod +x /srv/shared/t.sh && sudo -u ali /srv/shared/t.sh
sudo: unable to execute /srv/shared/t.sh: Permission denied
$ sudo visudo -f /etc/sudoers.d/webteam
## add: %webteam ALL=(root) /usr/bin/journalctl -u ssh*
$ sudo chmod 440 /etc/sudoers.d/webteam
$ sudo -l -U ali | tail -1
    (root) /usr/bin/journalctl -u ssh*
$ sudo reboot
$ findmnt -n -o OPTIONS /srv/shared  ## after reboot
rw,nosuid,nodev,noexec,relatime
```

✓ بررسی نتیجه (Verify)

بعد از ری‌استارت `/srv/shared` باید خودکار mount شده باشد. اجرای اسکریپت در آن با `Permission denied` رد می‌شود (به خاطر noexec)، اما `bash t.sh` همچنان کار می‌کند؛ پس noexec یک لایه‌ی دفاعی است، نه دیوار کامل. ali می‌تواند `sudo journalctl -u ssh` بزند ولی `sudo apt` برایش مجاز نیست.

تمرین‌ها و پرسش‌های مرور

پاسخ‌ها در پیوست «پاسخ‌نامه»

1. شش ستون fstab را به ترتیب نام ببرید.
2. چرا در fstab استفاده از UUID بهتر از `/dev/sdb1` است؟
3. گزینه‌های `noexec` و `nosuid` و `nodev` هر کدام چه چیزی را منع می‌کنند؟
4. چرا sudoers را فقط با `visudo` ویرایش می‌کنیم؟
5. علامت `%` در ابتدای یک خط sudoers چه معنایی دارد؟

جلد دوم

جلد دوم: از کاربر ترمینال تا مدیر سیستم

جلد اول شما را با خط فرمان آشنا کرد؛ این جلد آن را به ابزار کار روزانه‌تان تبدیل می‌کند. خروجی فرمان‌ها را به هم وصل می‌کنیم، با grep و Regex دقیقاً همان خطی را که لازم داریم بیرون می‌کشیم، متن را برش می‌دهیم و جایگزین می‌کنیم، با vim فایل تنظیمات ویرایش می‌کنیم و اولین اسکریپت‌های Bash را می‌نویسیم. بعد سراغ قلب سیستم می‌رویم: سخت‌افزار، فرایندها، سرویس‌ها، لاگ‌ها و شبکه، و در پایان کاربران و مجوزها؛ همه روی همان آزمایشگاه VMware و با لابراتوارهای سناریومحور.

در این جلد

- Pipe، Redirect و tee؛ grep و Regex
- آرشیو، cut و sort، sed، tr و uniq و awk
- ویرایشگرهای nano و vim، اسکریپت Bash
- سخت‌افزار، FHS، فرایند، systemd و لاگ
- شبکه، ip و ss، netplan و networkd
- کاربر و گروه، sudo، مجوزها، لینک و fstab

پس از این جلد می‌توانید

- ✓ خروجی فرمان‌ها را به هم وصل و فیلتر کنید
- ✓ از پروژه‌ها پشتیبان فشرده بگیرید
- ✓ فایل تنظیمات را با vim ویرایش کنید
- ✓ اسکریپت‌های Bash خودتان را بنویسید
- ✓ سرویس، لاگ و IP ثابت را مدیریت کنید
- ✓ کاربر بسازید و مجوزها را امن کنید



```
student@ubuntu-lab: ~

student@ubuntu-lab:~$ sudo apt install sysadmin-skills
Reading package lists... Done
Setting up sysadmin-skills (24.04-LTS) ...
student@ubuntu-lab:~$ echo "Volume 2 complete"
Volume 2 complete
student@ubuntu-lab:~$
```

NEXT · EXAM 010-160

آماده‌ی آزمون LPIC-1 و

مرور کادرهای «نکته‌ی آزمون»، پاسخ‌نامه و مسیر LPIC-1 ← LPIC-2 ← LPIC-3

مدرس و طراح لابراتوار

محمدرضا بیدختی

MOHAMMAD REZA BEIDOKHTI

UBUNTU 24.04 LTS

VMware Workstation 17+

LPI Linux Essentials 1.6